



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

MASTERS IN ARTIFICIAL INTELLIGENCE AND
DATA ENGINEERING

*Edge AI Computing for Grape Leaf Disease Detection in
LoRaWAN-Based Vineyard Monitoring System*

Advisories:

Prof. Vallati Carlo

Dr. Palla Fabrizio

Prof. Zinnai Angela

Candidate:

Tsegay Teklay Gebrelibanos
ID Number: 683925

Academic Year 2025/2026

Abstract

Existing vineyard monitoring systems primarily rely on environmental data collection such as temperature, humidity, and pressure, transmitted over LoRaWAN networks and providing limited insights into the actual grapevine's health status. This thesis work focuses on enhancing the vineyard monitoring intelligence by integrating Edge AI computing at sensor level for grape leaf disease detection. Moving beyond the traditional environmental-only monitoring approaches, this research was conducted on developing and optimizing deep learning models (CNN for classification and YOLO for detection) and deploying directly on resource-constrained edge sensors. This extension enabled the monitoring system to capture and process leaf images to detect diseases directly at the edge, providing real-time disease detection capabilities while adapting the inherent bandwidth and power limitations of LoRaWAN-based wireless sensor networks. This disease detection feature is implemented using four-stage pipeline architecture and deployed on ESP32-S3-microcontroller. **Stage 1** captures image of vine-trees using the OV3660 camera module operating at VGA resolution with JPEG compression. **Stage 2** employs an ESP-optimized YOLO11n model (ESPDet-Pico: 478KB INT8) to localize individual grape leaves within the captured frame, achieving 56.4% mAP@50 for leaf detection. Then these detected leaf regions are cropped and forwarded to **Stage 3**, where a MobileNetV2 classifier model (2.27 MB INT8) is utilized to identify disease symptoms across the four classes—Black Rot, Esca, Healthy, and Leaf-Blight with $99.8\% \pm 0.065\%$ accuracy on individual leaf patches. **Stage 4** aggregates the classification results from multiple detected leaves to produce a single output formatted for LoRaWAN transmission. Since each captured frame typically contains multiple grape leaves, the weighted aggregation algorithm combines detection confidence, classification entropy, and spatial quality metrics from up to 15 detected leaves to produce robust per-frame disease diagnoses result. The aggregated classification result is then packaged into LoRaWAN uplink packet (e.g. "Leaf Blight, 80% confidence") and transmitted over the WSNs.

To optimize also the power consumption, the system works on duty-cycle operation to capture and preprocess images and apply inference at 6-hour intervals and then enters into deep sleep mode. With a 2,400 mAh Li-SOC12 primary battery, the estimated operational lifetime is approximately 10 months under this duty-cycled regime. Overall, this thesis work demonstrated the feasibility of deploying deep learning models on resource-constrained microcontrollers specifically for disease detection in vineyard monitoring systems and precision agriculture applications at large.

Keywords: Wireless Sensor Networks, LoRaWAN, Precision Agriculture, Grape Leaf Disease Detection, Edge AI Computing, Object Detection Model, Image Classification, Quantization, ESP32-S3.



Acknowledgments

This thesis would not have been possible without the support, guidance, and generosity of many people. I would like to express what their contributions have truly meant to me.

First and foremost, I would like to express my sincere gratitude to my thesis advisor, **Prof. Carlo Vallati**. From the very beginning he believed in the potential of this project and gave me the freedom to explore ideas. His patience during long reviews, his sharp and to-the-point suggestions, his constructive feedback on every draft, and his enthusiasm for the work made all the difference. Under his guidance I have grown as an engineer.

My special thank you goes to my supervisor **Dr. Fabrizio Palla** for opening the door of **INFN** (Istituto Nazionale di Fisica Nucleare) to me. Offering me this internship was an act of trust that I will never forget. Working in his laboratory allowed me to bridge the gap between academic theory and hands-on experience with microcontrollers and sensor systems, and his mentorship taught me what it means to be responsive and punctual as a researcher.

I extend my special thank you to **Prof. Angela Zinnai** of the **Department of Agriculture** at the University of Pisa. Her deep knowledge of precision agriculture gave this project its real-world grounding, and her assistance in collecting the grape-leaf dataset was indispensable.

I would also like to express my sincere appreciation to **Bruno Casu** at INFN, who was always there when I hit a wall. Whether it was a hardware issue at the bench, a tricky software bug, or simply a moment of doubt, Bruno offered both his technical expertise and his calm encouragement. His day-to-day mentorship during the development and implementation phases was instrumental to the success of this project.

I gratefully acknowledge the **University of Pisa International Office** for awarding me the scholarship that made my studies in Italy possible. That opportunity changed the trajectory of my life, and I carry it with immense gratitude.

Finally, I would like to thank my friends **Kaleb Gebremicael**, **Gabriele Billi Ciani**, and **Gabriele Giudici**, who have accompanied me throughout all my study time here. Their thoughtful suggestions and constant encouragement during all the courses were invaluable. I am deeply grateful for their support and friendship.



Contents

Abstract	i
Acknowledgments	ii
List of Figures	vi
List of Tables	vii
List of Acronyms	viii
1 Introduction and Background	1
2 Literature Review	4
2.1 IoT & LoRaWAN Vineyard Monitoring Systems	4
2.2 Deep Learning Models for Grape Leaf Disease Detection	4
2.3 Lightweight and Edge-Based Grape Leaf Disease Detection Models	7
2.4 Research Gaps on the Existing LoRaWAN-based Vineyard Monitoring System	8
3 System Architecture and Model Development	10
3.1 System Overview and Methodology	10
3.1.1 Four-Stage Pipeline Architecture	10
3.1.2 Rationale for Two-Model Architecture	11
3.2 Dataset Preparation and Preprocessing	12
3.2.1 The Grape-Leaf Disease Dataset	12
3.2.2 Grapevine Dataset for Leaf Localization	13
3.3 Leaf Localization ESPDet-Pico Model (YOLO-based)	16
3.3.1 Object Detection Framework Selection	16
3.3.2 YOLO11-Optimization for Resources Constrained Deployment	17
3.3.2.1 Backbone Architecture Modifications	17
3.3.2.2 Detection Head Architecture	18



3.3.2.3	Parameter Reduction	19
3.3.3	Detection Performance Evaluation	21
3.3.4	Post-Training Quantization	22
3.3.5	Comparative Analysis on ESPDet-Pico vs YOLO11n	23
3.3.6	Deployment Feasibility and Trade-off Justification	24
3.4	Disease Classification Model (MobileNetV2)	24
3.4.1	Convolutional Neural Network Architecture	24
3.4.2	Transfer Learning Strategy	26
3.4.2.1	Stage 1: Classifier Head Training	26
3.4.2.2	Stage 2: Full Model Fine-tuning	26
3.4.3	MobileNetV2 Training and Optimization Techniques	27
3.4.4	Model Evaluation and Performance Metrics	28
3.4.5	Post-Training Quantization	29
3.4.6	MobileNetV2 vs ResNet18 Comparative Analysis	29
3.4.6.1	Accuracy and Model Size Trade-off Analysis	29
3.4.6.2	Architectural Implications for Edge Deployment	29
4	Edge AI System Implementation	31
4.1	Hardware and Development Framework	31
4.1.1	ESP32-S3 Microcontroller Architecture	31
4.1.2	ESP-IDF Development Framework	32
4.1.3	ESP-DL Inference Engine	32
4.2	Model Quantization and Format Conversion	33
4.2.1	The ESP-PPQ Quantization Framework	33
4.2.2	The ESPDet-Pico Model Quantization Pipeline	34
4.2.3	The MobileNetV2 Model Quantization Pipeline	35
4.2.4	Quantization Accuracy Preservation	36
4.3	Firmware Architecture and Memory Management	37
4.3.1	Model Storage and Loading Strategy	37
4.3.2	Initialization Order of Camera and Models	38



4.3.3	Buffer Allocation and Reuse Strategy	38
4.3.4	Memory Utilization Summary	39
4.4	Inference Pipeline Implementation	40
4.4.1	Camera Acquisition and JPEG Decoding	41
4.4.2	Leaf Localization and Coordinate Mapping	41
4.4.3	Leaf Region Cropping and Preprocessing	41
4.4.4	Disease Classification and INT8 Dequantization	42
4.4.5	Multi-Leaf Aggregation Strategies	43
4.5	Implementation Constraints and Design Trade-offs	44
5	Performance Evaluation and Discussion	46
5.1	Experimental setup	46
5.1.1	Hardware and Firmware Configuration	46
5.1.2	Test Environment	46
5.2	Experimental Inference Results	47
5.3	Inference Latency Analysis	48
5.4	Runtime Memory Utilization	49
5.4.1	Memory Footprint Analysis	49
5.4.2	Long-Term Stability	50
5.5	Power Consumption Analysis	50
5.5.1	Active and Sleep Power Measurements	50
5.5.2	Duty-Cycle Energy Analysis	51
5.6	Discussion	52
5.6.1	Trade-offs Between Accuracy, Latency, and Power	52
5.6.2	Limitations and Deployment Considerations	53
6	Conclusion and Future Work	54
6.1	Conclusion	54
6.2	Future Work	56
	Bibliography	57

List of Figures

1.1	LoRaWAN-based vineyard monitoring system architecture that only relies on environmental data for disease risk assessment	2
2.1	Image feature extraction to train IKNN model to detect grape leaf disease [1].	5
2.2	An Edge-boosted Crop Disease Detection System via TinyML and LoRa Communication [2].	8
3.1	System architecture of the proposed edge AI-based grape leaf disease detection LoRaWAN-based vineyard monitoring.	10
3.2	Four-stage edge AI pipeline for grape leaf disease detection.	11
3.3	Sample images from the grape leaf disease dataset, illustrating the four classes: Black Rot, Esca, Healthy, and Leaf Blight.	13
3.4	Annotated image from the leaf localization dataset, bounding boxes around individual grape leaves	14
3.5	Training curves for ESPDet-Pico on the grapevine dataset 3.2	21
3.6	ESPDet-Pico detection performance curves on the grape leaf test set. . .	22
3.7	Training and validation curves for the MobileNetV2 model.	28
4.1	ESPDet-Pico quantization pipeline from dataset preparation to ESP32-S3 deployment-ready INT8 model.	35
4.2	MobileNetV2 quantization pipeline from dataset preparation to ESP32-S3 deployment-ready INT8 model	36
4.3	Model storage and loading strategy for ESP32-S3 deployment. Both ESPDet-Pico and MobileNetV2 models are embedded in flash memory and loaded into PSRAM at runtime for inference execution.	37
4.4	Initialization sequence and PSRAM utilization for model loading and camera initialization.	39
4.5	End-to-end inference pipeline showing detection, cropping, and classification stages for grape leaf disease detection	40
4.6	The cropping and resizing process of the detected leaf region	42

List of Tables

3.1	Grape leaf disease dataset distribution across training, validation, test splits	12
3.2	Detection training augmentation configuration for ESPDet-Pico	15
3.3	ESPDet-Pico backbone architecture after	17
3.4	Comparative evaluation of ESPDet-Pico and YOLO11n baseline on grape leaf detection	23
3.5	Comparative analysis of MobileNetV2 and ResNet18	29
4.1	Memory utilization summary for ESP32-S3 deployment firmware	40
4.2	Implementation constraints and design trade-offs of deploying Edge AI system on ESP32-S3	45
5.1	Serial output from a single inference cycle	47
5.2	Per-stage latency breakdown for the inference pipeline (10 detected leaves)	48
5.3	End-to-end latency and throughput versus number of detected leaves . .	49
5.4	Detailed energy budget per 6-hour inference cycle	52

List of Acronyms

Acronym	Definition
AEC	Automatic Exposure Control
AI	Artificial Intelligence
AMP	Automatic Mixed Precision
API	Application Programming Interface
AWB	Automatic White Balance
BN	Batch Normalization
CIoU	Complete Intersection over Union
CNN	Convolutional Neural Network
DL	Deep Learning
DMA	Direct Memory Access
DRAM	Data Random Access Memory
DVP	Digital Video Port
ESP	Espressif Systems Processor
FLOPs	Floating Point Operations
FP	False Positive
FP32	32-bit Floating Point
FPN	Feature Pyramid Network
FPS	Frames Per Second
FreeRTOS	Free Real-Time Operating System
GAN	Generative Adversarial Network
GAP	Global Average Pooling
GPIO	General-Purpose Input/Output
GPU	Graphics Processing Unit
INT8	8-bit Integer Quantization
IoT	Internet of Things
IoU	Intersection over Union
IRAM	Instruction Random Access Memory
JPEG	Joint Photographic Experts Group
LLN	Low-Power Lossy Network
LoRaWAN	Long Range Wide Area Network
LPWAN	Low Power Wide Area Network
LRU	Least Recently Used
LSTM	Long Short-Term Memory

Acronym	Definition
mAP	mean Average Precision
MCU	Microcontroller Unit
ML	Machine Learning
MOSFET	Metal-Oxide-Semiconductor FET
NMS	Non-Maximum Suppression
NN	Neural Network
NVS	Non-Volatile Storage
ONNX	Open Neural Network Exchange
OTA	Over-The-Air
PIE	Processor Interface Extension
PPQ	Post-Processing Quantization
PSRAM	Pseudo-Static RAM
PTQ	Post-Training Quantization
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RGB	Red, Green, Blue
ROI	Region of Interest
RTC	Real-Time Clock
SCCB	Serial Camera Control Bus
SCDown	Spatial-Channel Downsampling
SGD	Stochastic Gradient Descent
SIMD	Single Instruction Multiple Data
SoC	System on Chip
SPI	Serial Peripheral Interface
SPPF	Spatial Pyramid Pooling Fast
SRAM	Static Random Access Memory
SVM	Support Vector Machine
TFLite	TensorFlow Lite
UART	Universal Asynchronous Receiver-Transmitter
USART	Universal Sync/Async Receiver-Transmitter
VGA	Video Graphics Array
WSN	Wireless Sensor Network
YOLO	You Only Look Once



Chapter 1

Introduction and Background

Cultivation of grapes is one of the most economically significant crops in the world, supporting wine production, juice, and fresh fruit markets [3]. However, as global cultivation expands, disease issues increasingly threaten yield and quality, destabilizing the industry [4, 3]. Grapevines are vulnerable to fungal and bacterial diseases including Black Rot, Esca, Leaf Blight, and others that can affect the entire harvest if not detected and managed promptly [3, 5, 6, 7]. Traditional disease identification relies on manual inspection by experts, which is laborious and vulnerable to error through human oversight [8]. In response to these limitations, The transition toward precision agriculture which is characterized by data driven management practices that has emerged as a critical strategy shift to address these challenges [9, 10]. Within this context, the development of intelligent monitoring systems capable of providing real time insights of plant health status become increasingly vital for vineyard management.

The existing vineyard monitoring system was developed as a long-range, low-power wireless sensor network (WSN) platform designed for environmental monitoring and management of vineyards, aiming to provide methods to evaluate and improve the energy consumption of sensor nodes. The goal of this monitoring system was to provide an efficient and reliable way to promptly detect conditions adverse to grapes by measuring environmental data such as temperature, pressure, soil moisture, and humidity levels of vineyard fields. As you saw in figure 1.1 the platform is composed of a set of data acquisition sensors scattered over large open field areas, a gateway node, and a remote server to process and store the acquired data. This monitoring platform is based on the LoRaWAN specification, which is suitable for long-range and low-power lossy networks (LLNs) applications and provides a reliable link layer protocol for the deployed end nodes (sensor nodes) to access and transmit data to a central gateway [11, 12]. The gateway node provides IP connectivity, forwarding the frames from the wireless sensor network to a local application server, which then sends the data to the LoRaWAN network server called The Things Stack. The telemetry data at the servers is gathered, stored in a MongoDB database, and displayed on a Grafana dashboard back to users, providing flexibility in the use and access of measured environmental conditions.

Although the existing LoRaWAN-based vineyard monitoring system showed high capability for long term data collection across large field areas using low power communication, a fundamental limitation is still existing in this current implementations [13, 9]. As some studies noted, systems with primarily rely on environmental data collection such as temperature, humidity, and pressure, transmitted over LoRaWAN networks and providing limited insights into the actual grapevine's health status [14]. Only environmental sensing provides indicators of disease risk rather than direct assessment of plant health status. This gap of LoRaWAN-based vineyard monitoring system, not visualizing the actual grape leaf status, represented as a significant barrier to achieve more intelligent and responsive vineyard management systems.

Meanwhile, deep learning models for plant diseases identification have showed remarkable

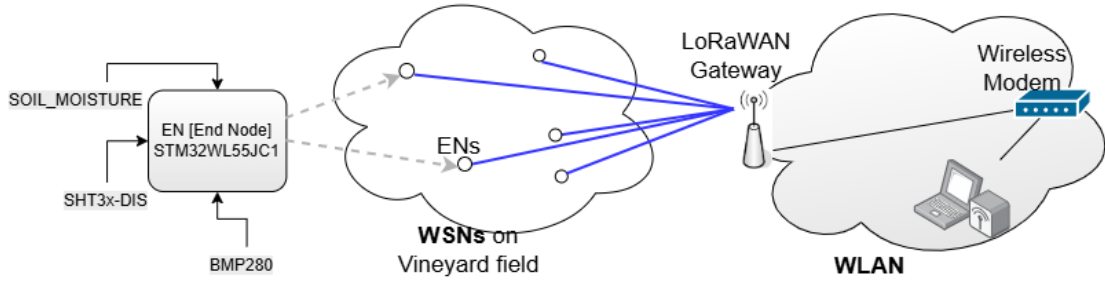


Figure 1.1: LoRaWAN-based vineyard monitoring system architecture that only relies on environmental data for disease risk assessment

performance in laboratory and controlled environments. This proved by a comprehensive review of deep learning for grape leaf disease detection [10]. This review shows convolutional neural networks and transformer based architectures can achieve high accuracy on detecting grape leaf diseases [15, 5]. Object detection frameworks such as YOLO also went further enabling automated leaf localization and disease symptoms in a complex field scenes [16, 17]. Despite these advancements, the practice of deploying these models in vineyard monitoring systems remained limited due two critical challenges. First, deep learning models typically demands high computational resources such as memory, processor and power, which is beyond the capabilities of the edge microcontroller [18]. Second, the LoRaWAN network’s payload is limited to 226 bytes. So transmitting raw image data is infeasible.

Edge computing and lightweight deep learning (TinyML) have become promising solutions to address computational resource and bandwidth limitations in LoRaWAN-based agricultural IoT systems. By performing image processing and inference at the sensor level, edge AI reduces the need for high-bandwidth data transmission. This lightweight vision models for disease detection in such resource-limited settings requires careful balance between model capacity and computational constraints [19, 20, 18]. However, most of the edge-based plant disease detection systems have been implemented on single-board computers such as Raspberry Pi or NVIDIA Jetson or smartphones, which offers much more computational resources than microcontrollers [21, 22, 2, 23]. Therefore, such kinds of edge computing are unsuitable for vineyard monitoring system which requires the edge computing for long term and battery powered deployment.

To address this gap and to further enhance the intelligence of the existing LoRaWAN-based vineyard monitoring system, this research introduced an edge AI extension feature focused on edge computing for grape leaf disease detection. The proposed extension Integrated edge AI at sensor level, by moving beyond the traditional environmental monitoring, for nearly real-time visual-based insights into grapevines. By processing leaf images and running CNN classification model at the edge sensor, the system improves its ability to identify grape diseases. This differentiates the monitoring system from other systems that only rely on environmental data, thereby providing a more intelligent and efficient solution for vineyard management. This visual-based disease detection required tactical use of Edge AI computing to overcome the bandwidth limitations inherent in LoRaWAN communication for transmitting image data. End-to-end inference pipeline was deployed on ESP32-S3 microcontroller, which is a low-power microcontroller with AI ca-

pabilities, enabling image acquisition, leaf localization, and disease classification directly at edge. The system transmit only compacted inference results e.g. “Leaf Blight” with confidence 90%, which is then seamlessly appended to the existing LoRaWAN payload. Besides, Duty-cycled operation was implemented to ensure effective power consumption, by setting 6-hour periodic intervals for image capture and on-device processing, followed by extended deep sleep periods. With a 2,400 mAh Li-SOCl₂ primary battery, the system achieves an estimated operational lifetime of approximately 10 months. Therefore, it aligns perfectly with a low-power, event-driven monitoring strategy, providing actionable insights while minimizing the impact on battery lifetime. Moreover, this comprehensive approach enhances the vineyard monitoring system by enabling early, localized detection of plant health issues.

The main aim of this research was to design, implement, and validate Edge AI system for grape leaf disease detection deployed on microcontroller within the existing LoRaWAN-based vineyard monitoring system. Specifically, this research introduces the direct visual-based disease detection into the existing environmental only monitoring system, by deploying a complete inference pipeline on ESP32-S3 microcontroller. Then the system integrates image acquisition, leaf localization within complex vineyard scenes, Multi-class disease classification, and inference results formatted to fit within LoRaWAN payload. Hence, contribution of this study have been diverse, addressing both technical and practical challenges of edge AI deployment for precision agriculture. It demonstrated feasibility of deploying these two-stage, object detection then classification pipeline on ESP32 microcontroller under strict computing resources constraints. This is a significant advancement beyond existing implementations that rely on powerful single-board computers or smartphone devices [21, 22, 23]. Also introduces a weighted aggregation on the classification results from multiple leaves within a single frame, to send a compacted and robust results over LoRaWAN communication.

Overall, this work provide a complete system architecture that filled the gap between environmental only monitoring and direct visual based disease detection. This research established a foundation for Intelligent precision agricultural systems that combine the reliability of LoRaWAN with optimized deep learning models deployed at the edge.

The thesis is organized in six chapters. Chapter 1 introduces the background of the research, outlining the research gap, main aim, and contributions of the study. Chapter 2 reviews related works in the fields of IoT and LoRaWAN for precision agriculture, deep learning for grape disease detection, edge computing for plant disease detection, and the research gap on the existing system. Chapter 3 describes the proposed system architecture and model design, including dataset preparation, the ESPDet-Pico leaf localization model, the MobileNetV2 disease classification model. Chapter 4 presents the edge AI deployment pipeline on the ESP32-S3 microcontroller, covering model quantization, firmware architecture, memory management, and inference pipeline implementation. Chapter 5 reports the performance evaluation and discussion, including latency analysis, accuracy validation, memory utilization, and power consumption analysis. Finally, Chapter 6 concludes the thesis by summarizing the findings, discussing implications, and outlining directions for future research.

Chapter 2

Literature Review

The literature review provides a comprehensive survey of relevant research in wireless sensor networks for agriculture, plant disease detection using computer vision, low-power communication protocols, and edge AI deployment on resource-constrained platforms. The review is organized thematically to highlight how prior work motivates the contributions of this thesis and to identify gaps that Edge AI-enabled visual monitoring addresses.

2.1 IoT & LoRaWAN Vineyard Monitoring Systems

The traditional vineyard monitoring systems are focused on environmental data acquisition through wireless sensor networks. Hnatiuc et al. developed an intelligent grapevine disease detection using IoT sensor network that integrates environmental sensors for monitoring temperature, humidity, and atmospheric conditions [9]. This approach represents the foundational methodology where IoT sensors collect environmental data to predict plant health status. Next, Gawande et al. proposed early prediction of grape disease occurrence using a hybrid classifier model in association with IoT sensors, demonstrating the integration of machine learning with IoT for environmental monitoring [13]. Their work emphasized that monitoring environmental parameters via wireless sensor networks enables the prediction of plant diseases [24].

The adoption of LoRaWAN technology represents a significant advancement for large-scale vineyard monitoring due to its nature in long-range, low-power characteristics. Kirubakaran and Samreen proposed a Grape Leaf Diseases Identification System Using Convolutional Neural Networks and LoRa Technology, which is an early attempt to combine the CNN diseases detection with LoRa network communication [25]. More recently, Hnatiuc et al. made a comprehensive study on IoT monitoring for early identification of diseases in grapevines, integrating AI with IoT sensor networks [14]. Their study revealed how the integration of AI with IoT sensor networks can enhance the diseases' detection in grapevines, but it was primarily focused on environmental data collection and did not explore the edge-based visual-based monitoring.

2.2 Deep Learning Models for Grape Leaf Disease Detection

The usage of deep learning techniques in grape leaf disease detection brought some improvement, evolving from the traditional machine learning approaches into sophisticated Neural Network architectures. Aher et al. provided comprehensive review on deep learning for grape leaf disease detection [10], by putting similar methods together and identifying key challenges in that domain. The early approaches were employed using the classical machine learning techniques, as demonstrated by Shantkumari and Uma, who used machine learning techniques for detection of grape leaf disease using SVM and IKNN [1]. Javidan et al. further explored diagnosis of grape leaf diseases using automatic K-means clustering and machine learning [26], achieved reasonable accuracy but with extensive

manual feature engineering.

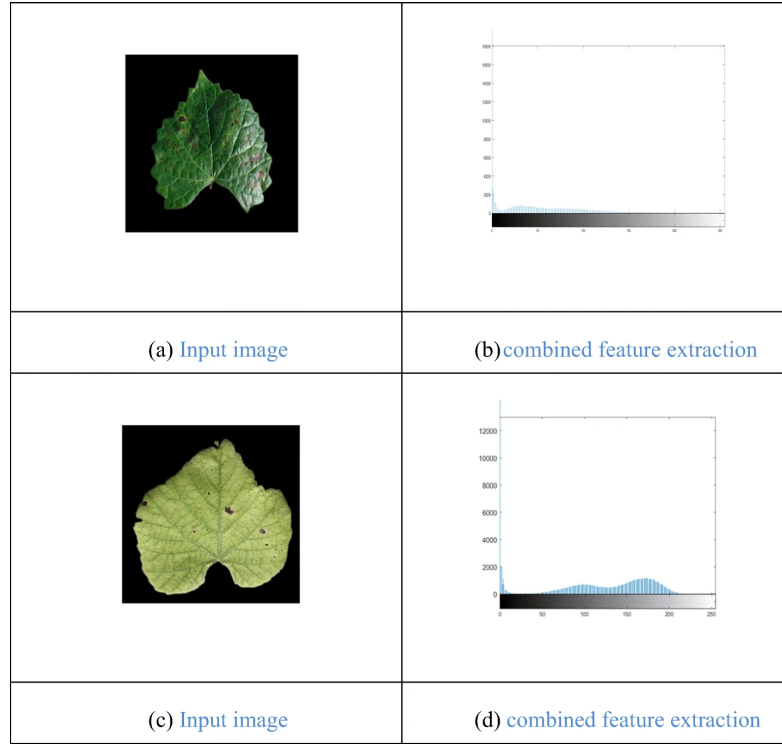


Figure 2.1: Image feature extraction to train IKNN model to detect grape leaf disease [1].

The transition to CNN showed a significant change in detection precision and automation. Diana Andrushia et al. introduced image-based disease classification on grape leaves using convolutional capsule network architectures[27], achieving accuracy of 99.12%. Nargotra and Sharma demonstrated grape leaf disease detection using deep learning with CNN-based approaches[28], while Soni explored deep learning for accurate grape disease classification in ResNet50 architecture[29]. The evolution towards the sophisticated architectures is evidence on Praveen et al.'s work on the novel classification approach for grape leaf disease detection based on different attention deep learning techniques[30], which applied attention mechanism to enhance feature descriptions.

Since the transfer learning became the dominant strategy to deal with not enough training data problems. Vo, Kheo and Nhon et al. together they proposed fusing Xception and ResNet50 features for robust grape leaf disease classification[31], demonstrating that combining several pre-trained architectures yields higher performance. Yadav et al. used the using VGG16 & Inception V3 models to investigate enhanced disease detection prediction accuracy of grape leaves[32], comparing deep learning architectures for classification tasks. Khursheed and Kaur presented improved grape leaf disease detection with GAN-augmented ResNet and EfficientNet [33], utilizing generative adversarial networks for data augmentation to address dataset imbalance.

The recent advancement have been focused on transformer-based architectures and hybrid ones. Zhang et al. developed a grape leaf disease recognition using the transformers [34], shows how capable transformer designs are architectures for disease recognition. Li et al. also came with the grape disease detection system using transformer's vision and environmental sensing[35], representing a novel approach that fuses visual and environ-

mental data streams. Karthik et al. suggested the idea of GrapeLeafNet which is a dual-track feature fusion network using inception-ResNet and shuffle-transformer for accurate grape leaf disease identification[15], achieving high accuracy scores because of its dual-track feature combination. Another further contribution done by Zhang et al. the cross-resolution transformer for grape leaf diseases segmentaion using context mining[36], demonstrating that cross-resolution transformer architecture design enables the precise disease segmentation.

There are some special architectures made for specific detection challenges. Talaat et al. Introduced a DeepLeaf, an optimized deep learning approach for automated recognition of grapevine leaf diseases[5], which optimize neural architectures for multi-class disease classification. Combining deep learning with manually created features, Atesoglu and Bingol suggested the detection and classification of grape leaf diseases with enhanced hybrid model based on feature engineering[7]. Combining deep feature extraction with traditional classifiers in hybrid techniques can produce competitive performance, as demonestrated by Ishengoma and Lyimo's ensemble model for grape leaf disease detection using CNN feature extractors and random forest classifier[37]. Using grape leaf disease identification with sparse data via GAN and CNN, Chen and Wu addressed data scarcity by means of GAN-based data augmentation to achieve the precision agriculture metrics[38].

The integrating object detection frameworks have made it possible to simultaneously localize and classify plant diseases in complex field environments. The YOLOv7 was used by Yang et al. to study the detection of small lesions on grape leaves [16], and he addressed the difficulty spotting the early-stage disease symptoms. Through architectural modifications, Chen et al. developed an enhanced lightweight grape leaf disease detection method [17], therefore improving it performance. Ji et al. carried out a research on grape leaf disease detection method based on NMA-YOLOv8n by introducing a novel attention mechanisms to YOLO architectures[39]. Particularly in relation to boundary detection issues in the situations of dense foliage, Zhou and Yu suggested a lightweight grape leaf disease identification for complex field scenes–YOLO-Grape model and its boundary-lesion coalesce optimization policies[6].

The next focus has been given to domain-specific challenges including multi-disease detection and disease severity evaluation. Mehta et al. investigated the grape leaf disease severity analysis using federated learning with CNN techniques[40], hence bringing privacy-preserving distributed learning for disease assessment. Combining spatial and temporal data for dynamic disease monitoring, Gaur et al. worked on fighting grape black rot with deep learning using CNN-LSTM hybrid model for disease severity classification[41]. In a review of advancements in deep learning for accurate classification of grape leaves and diagnosis of grape diseases[42], Kunduracioglu and Pacal chronicled the change from binary classification to multi-class, multi-disease detection systems. Many other contributions include work by Naik et al. on disease detection in grape leaves[43], Shetty and Shetty on grape leaf disease detection using deep learning approach"[44], and Liu and Li on grape leaf disease classification based on convolutional neural networks"[4]. These all contributions represent the ongoing evolution of deep learning techniques for grape leaf disease detection, with a clear trend towards more sophisticated architectures, multi-disease detection capabilities and consideration of real-world complexities.

2.3 Lightweight and Edge-Based Grape Leaf Disease Detection Models

The practical deployment of deep learning models for detecting grape leaf diseases in vineyard fields requires considering the constraints of available computational resources. Even though they achieved high performance in detecting grape leaf diseases, the deployment of these models on resource-constrained edge devices remains challenging. Hence, some researchers focused on developing lightweight neural network architectures specifically designed for edge deployment.

The deployment of deep learning models on the resource-constrained edge devices, at sensor level, is crucial step towards practical, real-time agricultural monitoring systems. Lin et al. started this research direction by developing a model named GrapeNet, a lightweight convolutional neural network model for identification of grape leaf diseases [45]. Their work demonstrated that optimized design of lightweight architectures while maintaining acceptable accuracy and significantly reducing computational requirements. This study established a foundation for "the lightweight convolutional neural networks" as a key design principle for edge deployment in resources-constrained applications.

Subsequently, research started to focus neural networks architectures designed and optimized for edge computing. Particularly MobileNet family, due to their depthwise separable convolutions and efficiently designed architectures to reduce computational complexity and make them suitable for resource-constrained devices. Sahid and Cahyadi developed image classification using MobileNet architecture for grape leaf disease detection[46], in a reduced parameter count and computational complexity. Meanwhile, Mathew et al. explored a real-time detection of grape leaf diseases with MobileNetV2 deep learning network[3], achieving real-time inference capabilities on edge hardware. Further improvements introduced attention mechanisms to MobileNetV2 architecture, while maintaining its lightweight characteristics, as shown by Dharrao et al.[8]. Hybrid approach, combining MobileNetV2 with transformer architectures have also been explored to increase performance while keeping the model lightweight[47].

Specialized lightweight architectures have been explicitly designed for resource-constrained edge device deployment. This research direction is initiated by Deepa et al. who introduced L-Net, lightweight CNN framework for sustainable multicrop leaf disease detection and classification on edge devices[48]. Their work highlighted importance lightweight architectural designs beyond MobileNet. Parallely, Zhang et al. explored a lightweight grape leaf disease recognition method based on transformer frameworks[34], demonstrating that even transformer architectures can also effectively compress for edge deployment without significant reduction in accuracy.

Since these works showed the effectiveness of lightweight designs, the integration of edge computing with practical deployment platforms has gained significant momentum. Tao et al. developed iCrop+, an edge-boosted crop disease detection system via TinyML and LoRa communication[2], demonstrating the integration of TinyML-based image segmentation on Raspberry Pi with LoRa communication for crop disease monitoring. Their system employs TinyML for edge-based image segmentation, followed by CNN inference on the segmented images, with results transmitted over LoRa communication, as shown

in figure 2.2. This work represents a significant step towards practical disease monitoring by integrating edge inference with low-power wireless communication, though deployment remains on single-board computers rather than microcontroller-class devices. Karim et al. also presented a landmark study on enhancing agriculture through real-time grape leaf disease classification via an edge device with a lightweight CNN architecture and Grad-CAM[21], demonstrating that lightweight CNN architectures can deploy and perform inference on resource-constrained edge devices. proceeding this work, Lazcano-García et al. developed a deep learning-based system for early symptoms recognition of grapevine red blotch and leafroll diseases and its implementation on edge computing devices[22], specifically addressing early disease detection on embedded platforms.

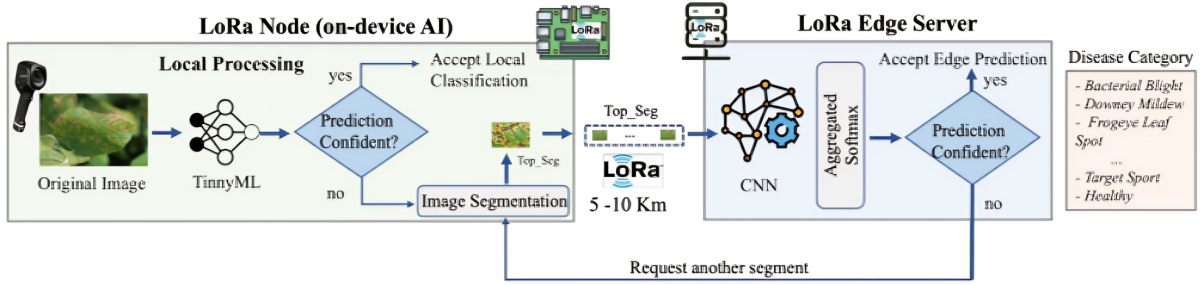


Figure 2.2: An Edge-boosted Crop Disease Detection System via TinyML and LoRa Communication [2].

More recently, some researchers start investigating microcontroller-level deployment, in the most resource-constrained scenario. Grujev et al.[49] demonstrated real-time apple disease classification approach using lightweight neural network on an Arduino edge device. Similarly, Amirkhanova et al.[50] developed a lightweight, end-to-end encrypted data pipeline for IoT. Here they explored ESP32 for secure communication in IoT, but they did not use it for visual based inference.

Another, model compression techniques have become crucial enablers for edge deployment. Quantization and pruning are common techniques to compress models. Joshi examined edge-AI for agriculture using lightweight vision models for disease detection in resource-limited settings[19], emphasizing that Edge AI systems requires careful balance between model accuracy and computational efficiency.

2.4 Research Gaps on the Existing LoRaWAN-based Vineyard Monitoring System

Despite the significant advancement progresses in both WSNs for vineyard monitoring and deep learning for grapevine disease detection, a crucial gap, combining them for practical, real-time disease detection still exists. Existing LoRaWAN-based vineyard monitoring systems, also demonstrated by Hnatiuc et al.[9], Gawande et al.[13], and Almufareh et al. [11], primarily focuses on environmental sensing. While these systems successfully transmit temperature, humidity, and atmospheric data over LoRaWAN networks, and they can provide indirect indicators of disease risk rather than direct visual based disease diagnosis.

Conversely, the substantial body of work on deep learning for grape leaf disease detection[15, 36, 5], they achieved high accuracy under unconstrained computational environments.

However, these models demands large computational resources, making them unsuitable for microcontroller level deployment.

Although lightweight and edge-based disease detection [21, 22, 48], have moved towards the embedded level deployment, like in Raspberry Pi, NVIDIA Jetson, and smartphones. However, these devices still consumes high power, so they cannot fit into the LoRaWAN-based vineyard monitoring system.

Moreover, the integration of complete inference pipelines, including image capturing, object detection, disease detection, and result aggregation on battery powered microcontroller within LoRaWAN networks remains unexplored. The existing efforts are either perform classification on pre-cropped images [25], rely on Raspberry Pi-level hardware [2], or assume smartphone-class resources [23].

Beyond the broader integration challenges, the technical deployment of both leaf localization and disease detection models into a single microcontroller is not investigated yet. Existing YOLO-based approaches for grape leaf detection[16, 17, 39], have focused on detection detecting the disease directly. They did not explore the integration of a two-stage pipeline where a lightweight model performs leaf localization followed by a separate classifier for disease classification. Similarly, Zhou and Yu[6] addressed leaf boundary detection in complex scenes but did not extend their work to additional model integration or wireless communication.

In summary, the core research gap addressed by this thesis is the lack of integrated, edge AI computing systems with capability of performing visual based grape leaf disease detection integrated into the existing LoRaWAN-based vineyard monitoring systems. Thus, this thesis work contributed in addressing this problem by developing Edge AI inference pipeline using ESP32-S3 microcontroller, camera, lightweight YOLO leaf detection model, MobileNetV2 multiclass disease classifier, under LoRaWAN communication constraints. Also duty-cycled operation with deep sleep power management is implemented to ensure long-term deployment feasibility. The following chapter presents the proposed system architecture and model design that addresses these identified gaps.

Chapter 3

System Architecture and Model Development

This chapter presents architectural design of the proposed Edge-AI system for grape leaf disease detection. The system is integrated into the existing LoRaWAN-based vineyard monitoring infrastructure as shown in Figure 3.1. The discussion covers the overall system design, including the four-stage inference pipeline, the reason behind the use of two distinct NN architectures for leaf localization and disease classification. In addition, the chapter includes dataset preparation, different preprocessing techniques, model training methodologies and performance optimization strategies. The detailed model architectures and training results are presented in Sections 3.3 and 3.4, while the hardware platform description and full deployment implementation are covered in Chapter 4.

3.1 System Overview and Methodology

The proposed system extends the existing LoRaWAN-based vineyard monitoring architecture by integrating edge AI computing capabilities for direct visual-based disease detection. The system is designed to operate within the resource constraints of ESP32-S3 microcontrollers while providing real-time disease diagnosis from captured grapevine images. The overall architecture is illustrated in Figure 3.1, showing how the edge AI components fit into the broader monitoring system.

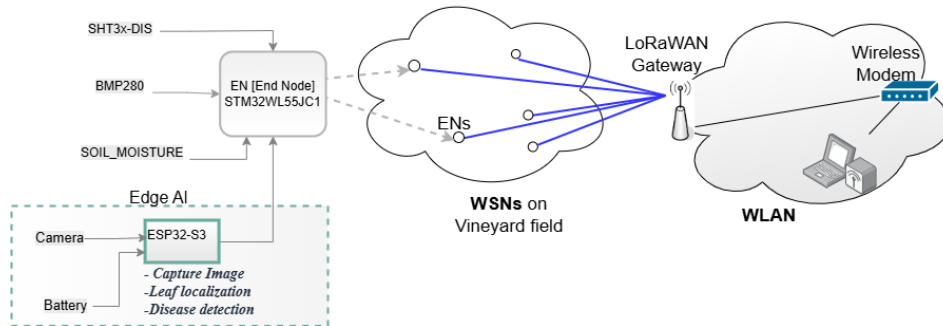


Figure 3.1: System architecture of the proposed edge AI-based grape leaf disease detection LoRaWAN-based vineyard monitoring.

3.1.1 Four-Stage Pipeline Architecture

The disease detection system is designed and implemented to operate in a four-stage pipeline and deployed on ESP32-S3 microcontrollers, as illustrated in Figure 3.2.

Stage 1: Image Acquisition - An OV3660 camera module capture grapevine images at VGA resolution (640×480). This module is designed to receive parallel video data signals, and its bus supports an 8-bit 16-bit DVP image sensor [51].

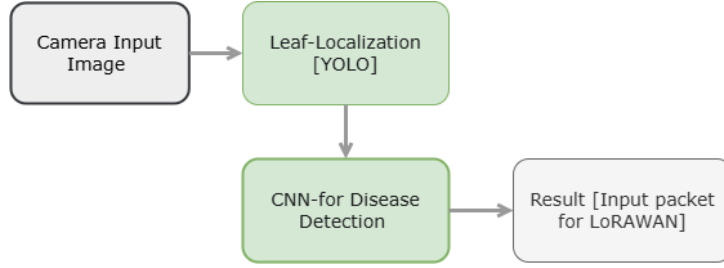


Figure 3.2: Four-stage edge AI pipeline for grape leaf disease detection.

Stage 2: Leaf Localization - A YOLO-based detection model (ESPDet-Pico) localizes individual grape leaves within the captured frame. This stage identifies and crops leaf, the regions of interest. The ESP-optimized model architecture and its training procedure are detailed in Section 3.3.

Stage 3: Disease Classification - Detected leaves at stage 2 are resized and classified by a MobileNetV2 model trained on different classes of grape disease single leaf images. The details of this model training, quantization, and evaluation are presented in Section 3.4.

Stage 4: Result Aggregation - Since a single frame contains multiple leaves, predictions should be aggregated using weighted combination of detection confidence, classification entropy, and spatial quality metrics. The aggregated result is encoded into LoRaWAN payload. This stage ensures that the final output is robust and informative for downstream decision-making while minimizing false positives and optimizing bandwidth usage in LoRaWAN transmission.

3.1.2 Rationale for Two-Model Architecture

The main purpose of the proposed system is grapevine disease diagnosis by analyzing their leaves. While the training dataset consists of single leaf images labeled with different disease classes. In real-world monitoring scenarios, the input images captured by camera contain multiple leaves in single frame. Therefore, training model with single leaf images to infer full frame is not effective. To address this, the system uses two distinct neural network architectures. One to localize and extract the target leaf regions from the image frame. The other to diagnose the disease symptoms on the extracted leaves.

Additionally, the leaf localization and disease classification tasks varies in nature and have different characteristics requiring specific architectures and training strategies. By separating these tasks into two specialized models, where each can be optimized for its specific task. The leaf localization model can focus on learning spatial features and bounding box regression to accurately identify leaf regions. Disease classification model can also concentrate on learning fine-grained visual features relevant to disease symptoms without being distracted by background noise. This modular design also allows for independent updates and improvements to each model as needed. For example, if a new disease class is added in the future, only the classification model needs to be retrained while the localization model can remain unchanged. This separation of concerns enhances the system's flexibility and maintainability while ensuring robust performance under real-world field conditions.

3.2 Dataset Preparation and Preprocessing

Two distinct datasets were used to train the leaf localization and disease classification models. The disease classification trained on dataset that contains single-leaf images on uniform backgrounds, enabling the model to learn disease-specific visual features without background interference. In contrast, the leaf localization trained on dataset that contains natural scenes of grapevine with annotated leaf bounding boxes, reflecting real deployment conditions characterized by occlusions, illumination variability, and background complexity.

In a real-world monitoring scenario, input images captured by camera contains multiple leaves in single frame with complex backgrounds and other environmental factors. So, during inference, localization model first detects leaf regions from full frame vineyard images. Then, the disease classification model diagnoses the disease symptoms on the extracted leaf. This task separation allows each model to specialize in its respective tasks while maintaining robustness under real-world field conditions.

3.2.1 The Grape-Leaf Disease Dataset

The grape-leaf disease dataset sourced from the PlantVillage Dataset hosted on Kaggle, created by Abdallah Ali [52]. PlantVillage is a widely adopted public repository of plant disease imagery developed through collaborative science efforts. The grape leaf subset aggregates pathology images captured under controlled laboratory conditions, with consistent lighting and uniform backgrounds. The dataset emphasizes leaf-level disease manifestations across four epidemiologically significant categories affecting the cultivation of grapevines worldwide. The dataset contains 4,062 RGB images distributed across four distinct disease classes: *Black Rot* (caused by *Guignardia bidwellii*), *Esca* (Black Measles, a complex disease involving multiple fungal pathogens), *Leaf Blight* (Isariopsis Leaf Spot, caused by *Pseudocercospora vitis*), and *Healthy* (disease-free grape leaf samples). Table 3.1 presents the complete class distribution statistics and train/validation/test splits. Sample images from each class are shown in Figure 3.3.

Table 3.1: Grape leaf disease dataset distribution across training, validation, test splits

Disease Class	Training	Validation	Test	Total
Black Rot	803	141	236	1,180
Esca (Black Measles)	940	166	277	1,383
Healthy	287	51	85	423
Leaf Blight	731	129	216	1,076
Total	2,761	487	814	4,062

The dataset uses a stratified partitioning with fixed 80/20 train-test split, allocating 3,248 images for model training and 814 images for final model testing. The training set is further subdivided into 85% training (2,761 images) and 15% validation (487 images). This split enables hyperparameter tuning on the validation set while preserving an independent test set for unbiased performance assessment at the end. Stratified sampling maintains class level proportion across all partitions, preventing from training bias toward majority classes.

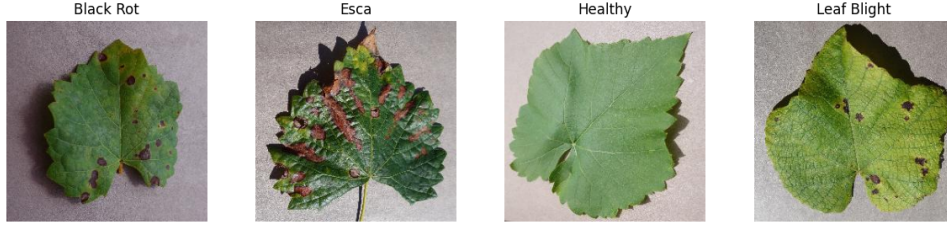


Figure 3.3: Sample images from the grape leaf disease dataset, illustrating the four classes: Black Rot, Esca, Healthy, and Leaf Blight.

Preprocessing Techniques

The input size for the classification model is set to 128×128 pixels, balancing model capacity with the memory constraints of the ESP32-S3 platform. The original images in the dataset are 256×256 pixels. To train the model, images are resized to 144×144 pixels and then randomly cropped to 128×128 pixels. But in validation and testing, it crops the center region of the resized image instead of random cropping. This allows the model to learn to focus on the most relevant leaf features while discarding edge pixels that may contain noise or artifacts. The random cropping during training also exposes the model to varied spatial contexts improving generalization.

All images are normalized using standard ImageNet preprocessing statistics (mean $\mu = [0.485, 0.456, 0.406]$ and standard deviation $\sigma = [0.229, 0.224, 0.225]$) as common practice in transfer learning with ImageNet pretrained models. Pixel intensities are first scaled to $[0, 1]$ by dividing by 255, then standardized via $x_{\text{norm}} = (x - \mu)/\sigma$. This normalization aligns input distributions with the pretrained MobileNetV2 network, facilitating effective transfer learning and fine-tuning convergence.

Training Dataset Augmentation

Data augmentation applied stochastically on-the-fly during each training epoch using PyTorch's `torchvision.transforms`, ensuring no two identical augmented samples are generated across epochs. This helps to prevent overfitting when dataset is not large enough to generalize well on unseen data. The augmentation pipeline of training set includes geometric and photometric transformations. Geometric augmentation include random horizontal and vertical flips, rotations, perspective distortions, and occlusions. Photometric augmentation include color jittering, grayscale conversion, and Gaussian blurring. These transformations are applied to simulate real leaf appearance cause by different lighting conditions, camera angles, and environmental backgrounds.

These augmentations applied with specific probabilities and ranges to balance the training dataset diversity while mimicking realistic leaf appearance. However, no augmentation is applied to validation and test sets. This ensures that model performance is evaluated on the original input data.

3.2.2 Grapevine Dataset for Leaf Localization

The leaf localization dataset comprises 531 natural vineyard photographs with manually annotated bounding boxes, enabling training of region-of-interest (ROI) detection models

for unconstrained deployment scenarios. Unlike the classification dataset that controlled laboratory conditions, these images capture real-world vineyard complexity. Images with variable leaf densities, overlapping foliage, occluded leaves, diverse lighting conditions, and cluttered backgrounds. The images are sourced from the Grape/Grapes dataset on Kaggle [53], curated by Nicolaas Regnier.



Figure 3.4: Annotated image from the leaf localization dataset, bounding boxes around individual grape leaves

Preprocessing Techniques

On this dataset preprocessing stage annotation, conversion to YOLO format, input resizing, and intensity normalization are applied. Image annotations were done manually with the assistance of the Segment Anything Model in the Roboflow platform. Each visible leaf instance in the vineyard images was centered with a bounding box and labeled as a single class (leaf). The bounding box coordinates were normalized to the YOLO format, which specifies each box via its center position (x_c, y_c) , width w , and height h , all scaled to the range $[0, 1]$ relative to the image dimensions. This normalization allows the model to learn scale-invariant representations of leaf regions, facilitating training across variable input sizes and resolutions. The complete annotated dataset, titled “Grape Leaf ROI Detection”, is now publicly available on the Roboflow platform [54] and partitioned into training (328 images, 64%), validation (82 images, 16%), and test (103 images, 20%) sets. The partitioning maintains natural distributions of leaf counts and scene complexity across splits, ensuring that the model is trained and evaluated on representative samples of real-world vineyard conditions. Figure 3.4 illustrates a sample annotated image from the dataset, showcasing bounding boxes around individual grape leaves in a complex vineyard scene.

The annotated dataset resizes input images to 416×320 pixels (width $\times$ height) for training the detection model. To fit the model’s input requirements while preserving aspect ratios, rectangular training is employed. This approach resizes images via letter-boxing, scaling them to fit within the target dimensions while maintaining their original

aspect ratio. The remaining space is filled with black padding, preventing distortion artifacts that would otherwise degrade detection accuracy on elongated or irregularly shaped leaves. This preprocessing strategy minimizes computational overhead on the ESP32-S3 device by avoiding complex resizing operations during inference, allowing the model to operate directly on the camera’s native output resolution.

For intensity normalization, YOLOv11 utilizes a linear scaling strategy where raw pixel intensities are mapped to the range $[0, 1]$ by dividing by the maximum 8-bit value: $x_{\text{norm}} = x_{\text{raw}}/255$. Unlike many deep learning architectures that require specific ImageNet-derived mean and standard deviation shifts, YOLO’s approach minimizes preprocessing overhead while remaining strictly consistent with its pre-training regime. This alignment ensures the preservation of learned features to transfer learning for specialized tasks, such as leaf detection.

Training Augmentation Pipeline

Leaf localization requires precise bounding box regression, necessitating a balanced augmentation strategy that enhances generalization without introducing excessive localization noise. Unlike classification, where aggressive color perturbation improves disease recognition, detection training prioritizes geometric robustness and leaf structure preservation. The augmentation pipeline is organized into three categories as photometric, geometric, and advanced compositing and summarized in Table 3.2.

Table 3.2: Detection training augmentation configuration for ESPDet-Pico

Category	Transform	Value
Photometric	Hue shift	± 0.015
	Saturation shift	± 0.4
	Value shift	± 0.4
Geometric	Rotation	$\pm 10^\circ$
	Translation	$\pm 10\%$
	Scale	$0.5\text{--}1.5\times$
	Perspective	0.0001
	Vertical flip	$p = 0.5$
	Horizontal flip	$p = 0.5$
Advanced	Mosaic	$p = 0.5$
	MixUp	$p = 0.05$
	Close mosaic	epoch 50

Photometric augmentations operate in the HSV color space. Hue variation is intentionally minimal (± 0.015) to preserve the characteristic green coloration that distinguishes foliage from background, while moderate saturation and brightness perturbations (± 0.4) account for natural lighting variability in vineyard environments.

Geometric augmentations simulate realistic camera positioning scenarios: rotation and shear approximate mounting angle deviations, translation models slight positional shifts, and scale variations. Both vertical and horizontal flips are applied with 50% probability. Perspective distortion is kept negligible (0.0001) to avoid unrealistic warping.

The advanced compositing techniques are YOLO specific augmentations designed for multi-scale robustness. Mosaic augmentation combines four training images into a single grid, improving small-leaf detection and multi-scale feature learning. MixUp probability is intentionally reduced to 5% (versus a typical 15%) because excessive pixel-wise blending degrades bounding box localization precision. CopyPaste augments effective dataset size by transferring leaf instances between images, improving occlusion handling. Mosaic is disabled after epoch 50 (`close_mosaic=50`) out of 1000 total epochs, allowing the remaining 950 epochs to fine-tune on unmodified images that more closely approximate real inference conditions.

Validation and test sets receive no augmentation, undergoing only letterboxing and normalization to ensure that performance metrics reflect true model capability on canonical inputs.

With both datasets prepared and preprocessing pipelines established, the following sections detail the design, training, and optimization of the leaf localization (Section 3.3) and disease classification (Section 3.4) models for embedded ESP32-S3 deployment.

3.3 Leaf Localization ESPDet-Pico Model (YOLO-based)

The first stage of the inference pipeline requires accurate localization and extraction of individual grape leaf regions from unconstrained vineyard imagery. This preprocessing step enables subsequent disease classification by isolating leaves from complex backgrounds, overlapping vegetation, and varying illumination conditions. Given the computational constraints of ESP32-S3 microcontrollers, the leaf localization model must satisfy two competing requirements: sufficient detection accuracy to reliably identify leaf boundaries, and minimal model complexity to enable real-time inference on resource-constrained hardware.

3.3.1 Object Detection Framework Selection

Modern object detection architectures can be broadly categorized into two paradigms: anchor-based two-stage detectors (e.g., Faster R-CNN [55]) and single-stage detectors (e.g., YOLO [56], SSD [57]). Two-stage detectors achieve high accuracy through region proposal networks followed by refinement stages, but it experienced with substantial computational overhead unsuitable for embedded deployment. Single-stage detectors directly predict class probabilities and bounding box coordinates in a single forward pass, offering superior inference speed at moderate accuracy. This is an advantageous trade-off for edge AI applications.

The YOLO (You Only Look Once) family [58] represents the state of the art in efficient single-stage detection. YOLO11n, the nano-scale variant developed by Ultralytics [58], serves as the baseline model for this work. YOLO11n employs an anchor-free detection paradigm, eliminating the need for predefined anchor boxes and simplifying post-processing. The architecture comprises a feature extraction backbone with cross-stage partial connections, a feature pyramid network (FPN) [59] for multi-scale fusion, and detection heads operating at three spatial resolutions to capture objects across varying scales.

YOLO11n achieves strong detection performance with 2.59 million parameters and a model size of 5.23 MB. However, these resource requirements exceed the 8 MB PSRAM capacity of the ESP32-S3 platform when accounting for activation memory, input buffers, and runtime overhead. Direct deployment of YOLO11n would necessitate model streaming from external flash storage, introducing prohibitive latency that precludes real-time operation. Consequently, aggressive model compression strategies are required to reduce memory footprint while preserving detection accuracy.

3.3.2 YOLO11-Optimization for Resources Constrained Deployment

To enable ESP32-S3 deployment, I used ESPDet-Pico, an ESP-optimized object detector derived from the YOLO11 architecture. ESPDet-Pico applies systematic architectural pruning, replacing computationally expensive operations with edge-deployable alternatives while maintaining the detection pipeline performance.

3.3.2.1 Backbone Architecture Modifications

The feature extraction backbone contains 11 layers organized into four downsampling stages, achieving spatial resolutions of P3/8, P4/16, and P5/32 relative to the input image. Table 3.3 details the complete backbone architecture after applying the compound scaling factors ($d = 0.50$, $w = 0.25$).

Table 3.3: ESPDet-Pico backbone architecture after

Index	Layer Type	Output Channels	Stride	Output Scale
0	Conv	16	2	P1/2
1	DSCnv	32	2	P2/4
2	ESPBlockLite	64	1	P2/4
3	DSCnv	64	2	P3/8
4	DSC3k2	64	1	P3/8
5	SCDown	64	2	P4/16
6	DSC3k2	64	1	P4/16
7	SCDown	128	2	P5/32
8	DSC3k2	128	1	P5/32
9	SPPF	128	1	P5/32
10	DSCnv [†]	128	1	P5/32

Depthwise Separable Convolutions (DSCnv): Standard convolutions are replaced with depthwise separable convolutions [60], factorizing spatial and channel-wise operations into a depthwise $k \times k$ filter followed by a pointwise 1×1 projection. This reduces computational complexity from $O(C_{in} \times C_{out} \times k^2)$ to $O(C_{in} \times k^2 + C_{in} \times C_{out})$, where k denotes kernel size. For typical layer configurations, this achieves 8–9× FLOPs reduction with minimal accuracy degradation.

ESP Blocks: ESPBlockLite and DSC3k2 modules employ hierarchical feature learning through parallel convolutional branches with varying dilation rates. Each branch captures multi-scale contextual information, which is subsequently fused via element-wise addition. Unlike standard residual blocks, ESP blocks maintain constant channel dimensions throughout the module, reducing memory bandwidth requirements.

Spatial-Channel Downsampling (SCDown): Stride-2 downsampling operations combine spatial pooling with channel projection in a single fused operation, reducing intermediate activation storage compared to sequential pooling-and-projection layers employed in YOLO11n.

Spatial Pyramid Pooling Fast (SPPF): The SPPF module, derived from the spatial pyramid pooling concept [61] and optimized in the Ultralytics framework [58], applies sequential 5×5 max pooling operations to capture multi-scale features efficiently. Compared to parallel pooling branches in the original SPP design, SPPF reduces memory allocation overhead by reusing intermediate pooled features.

3.3.2.2 Detection Head Architecture

The detection head contains functional components including a *neck* for fusing multi-scale backbone features. *prediction module* that generates per-scale bounding box and classification outputs.

Feature Pyramid Neck. The neck follows a bidirectional Feature Pyramid Network (FPN) design [59], fusing features from backbone stages P3 (1/8 scale), P4 (1/16 scale), and P5 (1/32 scale) to enable multi-scale detection. Feature fusion proceeds through three pathways:

1. **Top-down pathway:** P5 features are upsampled by $2\times$ via nearest-neighbor interpolation and concatenated channel-wise with P4 backbone features. The concatenated tensor is processed by two sequential ESPBlock layers (each containing a DS Bottleneck with residual connection), producing fused features at 1/16 scale.
2. **Second top-down stage:** The fused 1/16 features are upsampled by $2\times$ and concatenated with P3 backbone features, then processed by two ESPBlock layers to produce the small-object detection features at 1/8 scale.
3. **Bottom-up pathway:** A DSConv with stride 2 downsamples the 1/8 features and concatenates them with the 1/16 fused features from step 1, processed by two ESPBlock layers to produce medium-object features. A second stride-2 DSConv downsamples these and concatenates with the backbone layer 10 output, followed by two ESPBlock layers to produce large-object features at 1/32 scale.

This bidirectional design ensures that each detection scale benefits from both fine-grained spatial detail (via top-down flow from higher-resolution stages) and strong semantic context (via bottom-up flow from deeper stages).

Decoupled Prediction Heads At each of the three detection scales, two parallel prediction branches produce separate outputs for bounding box regression and object classification. This decoupled head design, inherited from YOLO11, separates localization and classification into independent sub-networks, avoiding the task conflict that arises when a single shared branch must simultaneously optimize for both objectives.

The *box regression branch*: at each scale consists of two consecutive DSConv layers followed by a 1×1 convolution that outputs 4 channels. Each channel corresponds to one edge distance (l, t, r, b) , representing the distances from the grid cell center to the left, top, right, and bottom edges of the predicted bounding box, respectively. This distance-based parameterization differs from the conventional center-size encoding (x, y, w, h) used in earlier YOLO versions and eliminates the need for anchor box priors, as bounding boxes are directly regressed from each spatial grid position in an anchor-free manner.

The *classification branch*: at each scale comprises two sequential blocks of depthwise convolution (DWConv) followed by a 1×1 pointwise convolution, terminated by a 1×1 convolution that outputs n_c channels (where $n_c = 1$ for the single leaf class). The output represents the class logit score for each grid cell.

Multi-Scale ONNX Export Format. For deployment on the ESP-DL inference runtime, the model is exported to ONNX format using a custom export path that produces six separate output tensors rather than the single concatenated tensor used in standard YOLO inference. Specifically, the exported model returns:

$$\text{Output} = \{(\mathbf{B}_s, \mathbf{S}_s)\}_{s \in \{P3, P4, P5\}} \quad (3.1)$$

where $\mathbf{B}_s \in \mathbb{R}^{1 \times 4 \times H_s \times W_s}$ contains raw box regression values and $\mathbf{S}_s \in \mathbb{R}^{1 \times 1 \times H_s \times W_s}$ contains raw class logits at scale s . Post-processing including sigmoid activation, distance-to-box coordinate conversion, and non-maximum suppression is deferred to the ESP-DL runtime on the microcontroller, avoiding the need for complex tensor reshaping and concatenation operations within the exported model graph.

3.3.2.3 Parameter Reduction

Through this systematic architectural pruning, ESPDet-Pico reduces the parameter count from 2.59 million (YOLO11n) to approximately 360,000 which is 86% reduction. Model size decreases from 5.23 MB to 1.06 MB in FP32 precision (79.8% reduction), enabling the model to fit entirely within ESP32-S3 PSRAM alongside activation buffers. Width scaling factor $w = 0.25$ reduces channel dimensions (e.g., base 512 channels become 128), while depth scaling factor $d = 0.50$ reduces internal block repetitions (e.g., base $2 \times \text{DSC3k2}$ becomes $1 \times$). This parameter reduction is achieved while maintaining YOLO11n architectural features that enable multi-scale feature learning and detection performance, as validated by the training results presented in the following section.

ESPDet-Pico Model Training and Optimization

With the compressed architecture established, training focuses on optimizing detection accuracy under the reduced model capacity constraint. ESPDet-Pico was trained on the grapevine leaf localization dataset described in Section 3.2, containing 328 training images, 82 validation images, and 103 test images. All images contain annotated bounding boxes for individual grape leaf instances in unconstrained vineyard scenes.

Input Resolution and Rectangular Training

The model accepts input images at 416×320 pixel resolution (width $\times$ height). Rectangular training is employed, where images within each batch are resized to maintain their original aspect ratio through letterboxing with black padding. This strategy reduces computational waste on padding pixels compared to square resizing and better preserves object proportions, particularly beneficial for elongated leaves in vineyard imagery.

Optimization Configuration

Training employed the AdamW optimizer [62] with weight decay regularization to prevent overfitting on the moderate-sized dataset. Key hyperparameters include:

- **Learning rate:** Initial learning rate $\eta_0 = 0.002$ with cosine annealing schedule [63] decaying to final learning rate $\eta_f = 0.00002$ (ratio $\eta_f/\eta_0 = 0.01$). Cosine annealing provides smooth learning rate decay without manual tuning of decay milestones.
- **Warmup strategy:** The first 3 epochs employ linear learning rate warmup from $\eta_{warmup} = 0.0001$ to the initial learning rate η_0 , stabilizing early training dynamics.
- **Momentum and weight decay:** Momentum coefficient $\beta_1 = 0.937$, weight decay $\lambda = 0.0005$.
- **Batch size:** 32 samples per batch, balancing training stability with GPU memory utilization.
- **Training duration:** Maximum 1000 epochs with early stopping patience set to 75 epochs, terminating training if validation mAP@50 does not improve for 75 consecutive epochs.

Mixed precision training (AMP) was enabled to accelerate training on GPUs with tensor cores, reducing memory footprint and training time without sacrificing numerical stability.

Loss Function

The training objective combines three loss components:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dff}}\mathcal{L}_{\text{dff}} \quad (3.2)$$

where $\mathcal{L}_{\text{box}}$ denotes the Complete IoU (CIoU) loss [64] for bounding box regression, $\mathcal{L}_{\text{cls}}$ represents the binary cross-entropy loss for objectness classification, and $\mathcal{L}_{\text{dff}}$ is the distribution focal loss for box coordinate refinement.

To prioritize localization accuracy over classification confidence in this single-class detection task (leaf versus background), the loss components are reweighted: $\lambda_{\text{box}} = 7.5$ (elevated for precise bounding box regression), $\lambda_{\text{cls}} = 0.5$ (reduced, as the model only distinguishes leaves from background), and $\lambda_{\text{dff}} = 1.5$ (standard weight for anchor-free detection). The elevated box loss weight emphasizes precise localization accuracy, critical for subsequent cropping operations in the inference pipeline.

Data Augmentation Strategy

The data augmentation pipeline for detection training is designed to enhance model robustness to the variability of real vineyard conditions while preserving the integrity of bounding box annotations. Balanced combination of geometric and mild photometric augmentations is applied stochastically during training, as summarized in Table 3.2, at section 3.3.

Training Convergence

Training converged after 949 logged epochs, with the best validation mAP@50 achieved at epoch 711. Figure 3.5 illustrates the training dynamics. The validation loss decreased smoothly without oscillations, indicating stable optimization. The cosine learning rate schedule is evident from the gradual decay curve. Early training (epochs 1 to 30) exhibits rapid performance improvement during the warmup phase, while subsequent training (epochs 30 to 700) demonstrates steady convergence as the model refines feature representations. After the best checkpoint at epoch 711, validation metrics plateau, triggering early stopping to prevent overfitting. The extended training duration (711 epochs) compared to YOLO11n’s faster convergence (327 epochs) reflects the reduced model capacity of ESPDet-Pico, requiring additional training iterations to reach optimal performance.

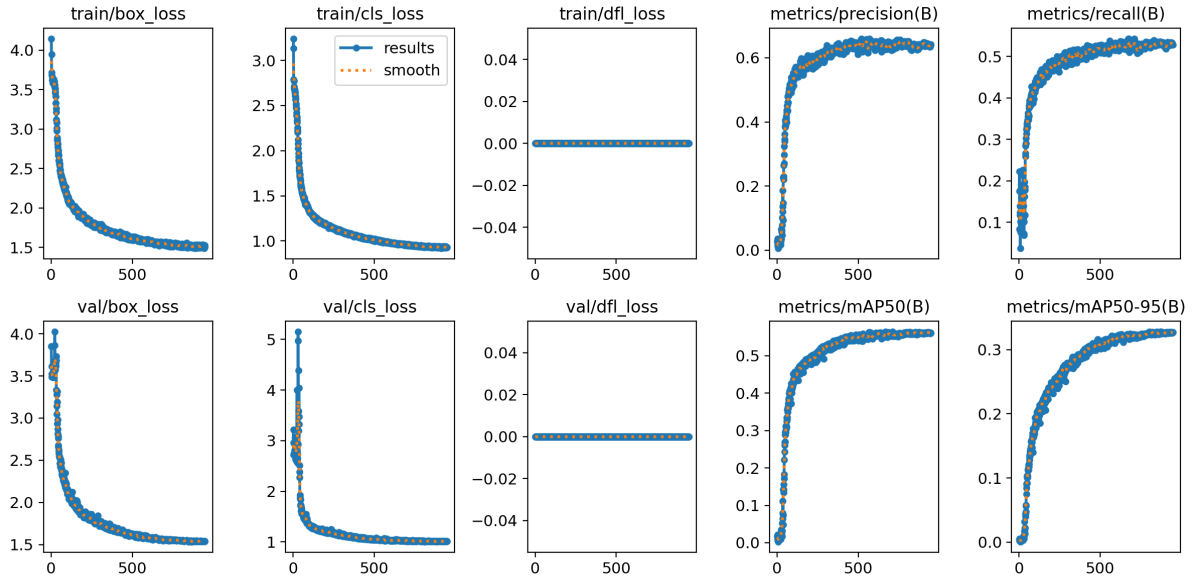


Figure 3.5: Training curves for ESPDet-Pico on the grapevine dataset 3.2

3.3.3 Detection Performance Evaluation

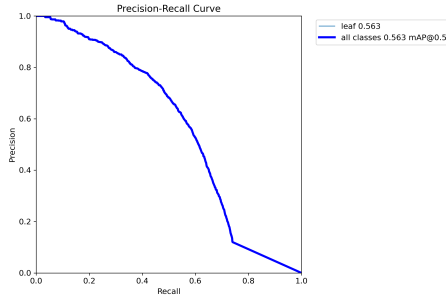
Model performance is evaluated using mean Average Precision at IoU threshold 0.5 (mAP@50), a standard metric for object detection [65]. mAP@50 computes the average precision across all classes, where a detection is considered correct if its Intersection over Union (IoU) with the ground truth bounding box exceeds 0.5.

ESPDet-Pico Performance Metrics

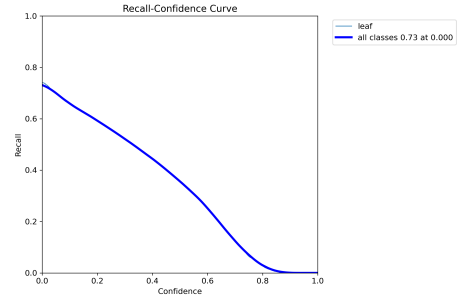
On the held-out test set (103 images), ESPDet-Pico achieves:

- **mAP@50:** 56.42%, demonstrating sufficient localization accuracy for leaf region extraction.
- **mAP@50-95:** 32.29%, reflecting performance across stricter IoU thresholds (0.5 to 0.95 in 0.05 increments).
- **Precision:** 62.77%, indicating controlled false positive rate.
- **Recall:** 54.17%, capturing the majority of visible leaves while maintaining precision.

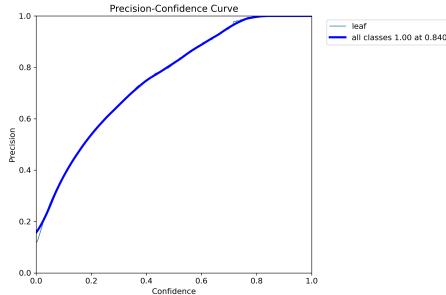
The precision-recall balance favors precision over recall, aligning with the two-stage pipeline design: false negatives (missed leaves) result in lost disease detection opportunities, while false positives (spurious detections) trigger unnecessary classification inference, wasting computational resources. The elevated precision ensures that detected regions predominantly contain leaves rather than background artifacts.



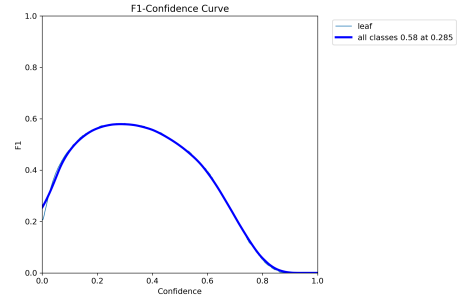
(a) PR curve ($\text{mAP}@0.5 \approx 0.563$).



(b) Recall vs confidence (max ≈ 0.73 at $\text{conf} = 0$).



(c) Precision vs confidence ($P \approx 1.0$ at $\text{conf} \approx 0.898$).



(d) F1 vs confidence (best F1 ≈ 0.57 at $\text{conf} \approx 0.248$).

Figure 3.6: ESPDet-Pico detection performance curves on the grape leaf test set.

3.3.4 Post-Training Quantization

To enable deployment on the ESP32-S3, ESPDet-Pico undergoes post-training quantization to INT8 precision using the ESP-PPQ framework [66], following the same quantization pipeline described for MobileNetV2 in Section 4.2.1. The quantized model size reduces from 1.06 MB (FP32) to 478 KB (INT8), a 55% reduction. The complete quantization procedure, including calibration strategy and ONNX export, is detailed in Chapter 4.

3.3.5 Comparative Analysis on ESPDet-Pico vs YOLO11n

This section compares the standard YOLO11n and the ESP-optimized YOLO11 architecture (ESPDet-Pico). Both models trained with identical configurations and datasets to understand the impact of using the ESPDet-Pico architecture over the pure YOLO11n. Table 3.4 presents the performance comparison.

Experimental setup for both architectures

for a comparison, both models trained on the same configuration:

- **YOLO-format dataset:** (train/val/test splits)
- **Image size:** 320×416 pixels
- **Batch size:** 32
- **Optimizer:** AdamW (lr=0.002, weight decay=0.0005)

Model detection performance and size comparison

Table 3.4: Comparative evaluation of ESPDet-Pico and YOLO11n baseline on grape leaf detection

Metric	YOLO11n	ESPDet-Pico	Difference	Relative Change
mAP@50 (%)	61.75	56.42	−5.33	−8.6%
mAP@50-95 (%)	43.57	34.29	−9.28	−21.22%
Precision (%)	69.55	64.77	−4.78	−6.8%
Recall (%)	57.29	54.17	−3.13	−5.5%
Parameters	2.59M	~0.36M	−2.23M	−86.1%
Model Size	5.23 MB	1.06 MB	−4.17 MB	−79.8%
Best Epoch	327	711	+384	+117.4%

YOLO11n outperforms ESPDet-Pico across all accuracy metrics, as expected given its substantially higher model capacity. However, the performance gap remains moderate: ESPDet-Pico retains 91.4% of YOLO11n’s mAP@50 while reducing model size by 79.8%. The mAP@50-95 gap is larger (25.9% relative drop), indicating that ESPDet-Pico struggles with tight localization at high IoU thresholds, but maintains acceptable coarse localization (IoU > 0.5) sufficient for leaf cropping.

The extended training duration for ESPDet-Pico (711 epochs vs 327 for YOLO11n) reflects the reduced representational capacity. The compressed model requires more training iterations to converge to comparable performance. This suggests that ESPDet-Pico operates near the lower bound of model capacity for this task—further pruning would likely yield diminishing returns or unacceptable accuracy degradation.

Overall, This comparison shows a trade-off between detection accuracy and model size, which is crucial for the resource-constrained devices. Therefore, the 5.33% accuracy reduction in ESPDet-Pico represents an acceptable trade-off for enabling edge deployment on ESP32-S3 device with significant model size reduction.

3.3.6 Deployment Feasibility and Trade-off Justification

The 79.8% model size reduction enables full in-memory deployment on ESP32-S3 without external storage access. The 1.06 MB ESPDet-Pico model, combined with activation buffers (approximately 1–2 MB), fits comfortably within the 8 MB PSRAM budget while reserving memory for the classification model and intermediate buffers. In contrast, YOLO11n’s 5.23 MB footprint would exceed available memory when combined with activation buffers, necessitating model streaming from flash storage and incurring prohibitive latency that precludes real-time operation.

The 5.33 percentage point mAP@50 reduction represents an acceptable trade-off for real-time edge deployment. At 56.42% mAP@50, ESPDet-Pico successfully localizes the majority of visible leaves, providing sufficient region proposals for downstream classification. Missed detections (false negatives) are partially mitigated by temporal redundancy in video stream processing: leaves undetected in one frame may be detected in subsequent frames as camera position or leaf orientation changes. False positives are controlled through precision-weighted aggregation in the multi-leaf inference stage (Chapter 5).

Furthermore, leaf localization task exhibits less strict accuracy requirements compared to the subsequent disease classification stage. Imperfect localization (e.g., bounding boxes that slightly truncate leaf edges or include small background regions) does not highly affect classification performance, as MobileNetV2. The primary objective—isolating leaves from complex backgrounds—is achieved reliably by ESPDet-Pico despite its compressed architecture.

3.4 Disease Classification Model (MobileNetV2)

Following the leaf localization, each detected leaf region undergoes disease classification to diagnose if the leaf is affected by pathology. This stage requires high classification accuracy to ensure reliable disease diagnosis while maintaining computational efficiency for embedded deployment. The classification model must generalize well across grape disease categories despite the moderate training dataset size, while fitting within the constrained memory allocation and power consumption requirements. Hence, a lightweight convolutional neural network architecture with proven transfer learning capabilities is selected as the backbone for disease classification.

3.4.1 Convolutional Neural Network Architecture

The disease classification model is based on MobileNetV2 [67], a lightweight convolutional neural network architecture specifically designed for resource-constrained mobile and embedded devices. MobileNetV2 achieves computational efficiency due to its architectural design. It is designed with a focus on reducing the number of parameters and floating-point operations (FLOPs). The key architectural design elements that enable MobileNetV2’s efficiency are the use of depthwise separable convolutions and the introduction of inverted residual blocks with linear bottlenecks.

Depthwise Separable Convolutions

Standard convolutional layers perform joint spatial and channelwise filtering, obtaining computational cost proportional to $C_{in} \times C_{out} \times k^2 \times H \times W$, where C_{in} and C_{out} denote input and output channels, k is the kernel size, and $H \times W$ represents the spatial dimensions. MobileNetV2 factorizes this operation into two sequential steps in the following manner:

1. **Depthwise convolution:** Applies a single $k \times k$ filter independently to each input channel, producing C_{in} feature maps. Computational cost reduces to $C_{in} \times k^2 \times H \times W$.
2. **Pointwise convolution:** Projects the depthwise features into C_{out} output channels using 1×1 convolutions, costing $C_{in} \times C_{out} \times H \times W$.

The total computational cost becomes $(C_{in} \times k^2 + C_{in} \times C_{out}) \times H \times W$. For typical configurations where $C_{out} \gg k^2$, this factorization achieves a lot of FLOPs reduction compared to standard convolutions. Depthwise separable convolutions significantly reduce the number of parameters and computational cost while maintaining representational capacity, making them ideal for mobile and embedded applications. However, they may introduce a slight accuracy drop compared to standard convolutions, which is mitigated through careful architectural design and training strategies.

Inverted Residual Blocks

Traditional residual blocks [68] employ wide-narrow-wide channel configurations, where intermediate layers have fewer channels than input/output layers to reduce computation. MobileNetV2 inverts this design, using narrow-wide-narrow blocks that expand the channel dimension in intermediate layers through 1×1 convolutions, apply depthwise convolution in the high-dimensional space, and then project back to a narrow output via 1×1 linear projection without activation (linear bottleneck). This architecture enables efficient feature transformation while maintaining low-dimensional bottleneck representations suitable for memory-constrained devices.

Skip connections are applied only when input and output dimensions match, allowing gradient flow and facilitating training convergence. The linear bottlenecks prevent information loss from ReLU activation [67], which would otherwise zero-out negative values in low-dimensional spaces. The combination of depthwise separable convolutions and inverted residual blocks allows MobileNetV2 to achieve a favorable accuracy-efficiency trade-off, making it a strong candidate for the disease classification task in this project.

Model Configuration and Pre-training

The MobileNetV2 backbone is initialized with weights pre-trained on ImageNet [69], a massive visual database containing over 14 million high-resolution images across 1000 object categories. Pre-training provides generic low-level and mid-level visual features transferable to the grape disease classification domain, accelerating convergence and improving generalization compared to random initialization.

The input resolution is set to 128×128 pixels, to fit within the memory capacity constraint. The original input image size was 256×256 pixels, but it was reduced to 128×128 to reduce the computational load and memory footprint of the model. However, the controlled nature of the PlantVillage dataset enables effective disease recognition at reduced resolution. The reduced input size also decreases memory footprint for runtime activations during inference.

The classification head consists of a global average pooling layer that aggregates the $4 \times 4 \times 1280$ feature maps (resulting from the 128×128 input resolution downsampled by a stride factor of 32 through the network) into a 1280 dimensional feature vector, followed by a dropout layer (dropout rate $p = 0.5$) for regularization, and a final fully connected layer with 4 output units (one per disease class). The dropout layer prevents overfitting by randomly deactivating 50% of neurons during training, forcing the network to learn robust distributed representations rather than over memorizing. The complete forward pass through the adapted MobileNetV2 architecture is summarized as:

$$128 \times 128 \times 3 \xrightarrow{\text{backbone}} 4 \times 4 \times 320 \xrightarrow{1 \times 1 \text{ conv}} 4 \times 4 \times 1280 \xrightarrow{\text{GAP}} 1280 \xrightarrow{\text{FC}} 4 \quad (3.3)$$

3.4.2 Transfer Learning Strategy

Adapting ImageNet pre-trained weights to the grape disease classification domain requires careful fine-tuning to leverage generic visual features while learning grape leaf disease specific patterns. A two-stage progressive fine-tuning strategy is employed to balance feature transfer (Transfer Learning) with domain specific specialization (Fine-tuning).

3.4.2.1 Stage 1: Classifier Head Training

The first stage focuses on training the randomly initialized classification head while keeping the pre-trained MobileNetV2 backbone frozen. This gives stable and informative features to the classifier head. By freezing the backbone, the classifier head can rapidly adapt to the specific disease categories. This is a fine-tuning with high learning rates on a small domain-specific dataset destroys useful pre-trained representations [70]. During this stage, only the global average pooling, dropout, and fully connected layers receive gradient updates.

In a few epochs with initial learning rate $\eta_1 = 10^{-3}$. The relatively high learning rate enables rapid adaptation of the classification head to the four grape disease categories. Few epochs are sufficient for the classifier head to learn, while preventing overfitting. The frozen backbone ensures that the generic visual features learned from ImageNet are preserved, providing a strong foundation for subsequent fine-tuning.

3.4.2.2 Stage 2: Full Model Fine-tuning

The second stage unfreezes all layers and fine-tunes the entire network. This stage allows the backbone to adapt its training dataset features. It refines the learned feature according to the specific visual characteristics of grape diseases leaf patterns.

Unlike the first stage, fine-tuning requires much more epochs with reduced learning rate $\eta_2 = 10^{-4}$. The $10\times$ learning rate. The reduction compared to Stage 1 ensures stable

fine-tuning of pre-trained weights without destabilizing learned representations. The sufficient number of epochs provides sufficient iterations for convergence while preventing overfitting, as validated through early stopping on the validation set.

3.4.3 MobileNetV2 Training and Optimization Techniques

Optimization and other Hyperparameters Configuration

Both training stages employ the AdamW optimizer [62], which combines adaptive learning rates with decoupled weight decay regularization. AdamW decouples the weight decay term from the gradient-based update, applying L2 regularization directly to weights rather than through gradient computation. This modification improves generalization compared to standard Adam [71] by preventing weight decay from interfering with adaptive learning rate scaling. Hyperparameters are configured to balance convergence speed with generalization performance. The learning rate is set to $\eta_1 = 10^{-3}$ for Stage 1 and reduced to $\eta_2 = 10^{-4}$ for Stage 2, enabling coarse adaptation of the classifier head before fine-grained tuning of the full network. Weight decay is configured at $\lambda = 10^{-4}$ as the L2 regularization coefficient, and the momentum parameters retain the AdamW defaults ($\beta_1 = 0.9$, $\beta_2 = 0.999$). A batch size of 32 samples is used throughout both stages, balancing training stability with GPU memory utilization.

Loss Function

Training minimizes the cross-entropy loss for multi-class classification:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}) \quad (3.4)$$

where N denotes batch size, $C = 4$ is the number of disease classes, $y_{ic} \in \{0, 1\}$ represents the ground truth label (one-hot encoded), and $\hat{y}_{ic} = \text{softmax}(z_c)$ is the predicted probability for class c of sample i . Cross-entropy loss penalizes confident incorrect predictions more heavily than near-boundary errors, encouraging the model to learn discriminative class boundaries. The main purpose of loss function is to optimize model parameters during evaluation.

On-the-fly Data Augmentation

Beyond weight decay and dropout, the data augmentation pipeline described in Section 3.2 provides implicit regularization by presenting diverse augmented variations of each training sample. The combination of geometric transformations (rotation, scaling, translation) and photometric perturbations (color jitter, brightness adjustment) exposes the model to realistic deployment variability, improving robustness to lighting changes, leaf orientation variations, and camera perspective differences encountered in vineyard environments.

Early Stopping and Model Checkpointing

Early stopping monitors validation loss with patience set to 10 epochs. Training terminates if validation loss does not improve for 10 consecutive epochs, preventing overfitting

beyond the optimal checkpoint. The model snapshot with lowest validation loss across all epochs is retained as the final model.

Multi-Seed Experimentals

To assess model stability and generalization consistency, training is repeated with four different random seeds: 42, 123, 456, and 789. Each seed initializes the random number generator controlling parameter initialization, data shuffling, and stochastic augmentation. This protocol quantifies performance variance attributable to random initialization rather than systematic dataset characteristics, providing confidence intervals for reported accuracy metrics [72].

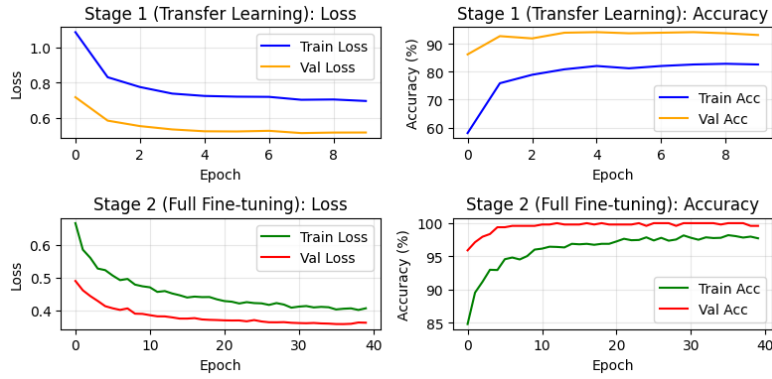


Figure 3.7: Training and validation curves for the MobileNetV2 model.

Figure 3.7 illustrates the training dynamics on of the experimental runs(seed 789), showing robust convergence across both stages. In (Fine-tuning) further refines the model, reducing validation loss to approximately 0.36 and reaching the accuracy of 99.88%. Throughout training, validation loss remains lower than training loss, indicating effective regularization via data augmentation and dropout, preventing overfitting to the training set.

3.4.4 Model Evaluation and Performance Metrics

The fine-tuned MobileNetV2 model achieves high classification accuracy on the unseen test set (814 images), demonstrating strong generalization despite the moderate training set size. Aggregated results across four independent training runs yield the following performance metrics:

- **Test Accuracy:** $99.80\% \pm 0.065\%$ (mean $\pm$ standard deviation)
- **Macro F1-score:** 0.9980 ± 0.0010
- **Macro Precision:** 0.9982 ± 0.0015
- **Macro Recall:** 0.9979 ± 0.0006

The low standard deviation ($\sigma = 0.065\%$) across random seeds indicates robust training stability, with minimal sensitivity to weight initialization. All four runs converge to nearly the same performances (99.75%, 99.75%, 99.88%, 99.88%), validating the reliability of the training protocol and the effectiveness of the fine-tuning strategy.

3.4.5 Post-Training Quantization

To enable deployment on the ESP32-S3 microcontroller, the trained MobileNetV2 model must undergo post-training quantization (PTQ) to convert floating point weights and activations to 8-bit integers (INT8). PTQ reduces model size from 8.48 MB to 2.27 MB (INT8), a 73.2% compression, while maintaining test accuracy 99.63%. This robustness is attributable to MobileNetV2’s batch normalization layers and inherently quantization friendly inverted residual architecture [73]. The detailed quantization pipeline, including calibration procedures, equalization strategies, and the ESP-PPQ framework configuration, is presented in Section 4.2 of Chapter 4.

3.4.6 MobileNetV2 vs ResNet18 Comparative Analysis

A comparative evaluation was conducted between MobileNetV2 and ResNet18 [68], both configured identically with 128×128 input resolution and trained under the same two stage fine-tuning protocol (Stage 1: 10 epochs at $\eta_1 = 10^{-3}$, Stage 2: 40 epochs at $\eta_2 = 10^{-4}$, AdamW optimizer, batch size 64, weight decay $\lambda = 10^{-4}$). Table 3.5 summarizes the comparative results in terms of model size and classification performance on the test set.

Table 3.5: Comparative analysis of MobileNetV2 and ResNet18

Metric	MobileNetV2	ResNet18
Model Size (FP32)		
ONNX file	8.48 MB	44.7 MB
PyTorch checkpoint	13.4 MB	46.8 MB
Classification Performance		
Test accuracy	99.80%	99.88%
Macro F1-score	0.9980	0.9990
Macro precision	0.9982	0.9991
Macro recall	0.9979	0.9989

3.4.6.1 Accuracy and Model Size Trade-off Analysis

Although ResNet18 achieves slightly higher accuracy, Both models perform well on the test set. The 0.08% accuracy gap is negligible in practice. However, the model size difference is significant, which has critical implications in deployment on memory-constrained microcontroller platforms. ResNet18 is approximately $5.3\times$ larger than MobileNetV2 in ONNX format and $3.5\times$ larger in PyTorch checkpoint format.

3.4.6.2 Architectural Implications for Edge Deployment

The architectural differences between MobileNetV2 and ResNet18 have profound implications in resource constrained environments. MobileNetV2 is designed for mobile and embedded applications, utilizing depthwise separable and residual blocks to minimize parameter count. In contrast, ResNet18 is a general-purpose architecture optimized for high accuracy on large-scale datasets. Therefore, the architectural design of MobileNetV2 is inherently suitable for microcontroller-class hardware like the ESP32-S3.



To summarize, MobileNetV2 was chosen as the disease classification model for the following reasons. Primarily, its architectural design, it can quantize into small size (2.27 MB) and fit to the PSRAM budget. This allowed the concurrent deployment with ESPDet-Pico detector for the complete two-stage pipeline. Secondly, its quantization-friendly architecture, with batch normalization and inverted residuals, maintains high accuracy under aggressive INT8 quantization. Finally, its lower computational cost is crucial for battery powered sensor nodes.

Chapter 4

Edge AI System Implementation

In this chapter, the complete embedded implementation of the Edge AI grape leaf disease detection system on ESP32-S3 microcontroller is presented. Following the chosen models design and development described in Chapter 3, this chapter addresses the critical steps of deploying two neural network models on a resource-constrained microcontroller particularly ESP32-S3.

This chapter begins with a description of the chosen hardware and its development framework, including the ESP-IDF development framework, and the ESP-DL inference engine presented in Section 4.1. Section 4.2 details the procedures of post-training quantization pipeline that converts both models from floating point to INT8 integer representations compatible with the ESP-DL inference engine. Section 4.3 discusses firmware architecture and memory management strategies, including challenges related to model storage, initialization sequence, and buffer allocation. Section 4.4 demonstrates complete inference pipeline implementation, from image capturing through inference results aggregation. Finally, Section 4.5 documented the key implementation constraints and design trade-offs.

4.1 Hardware and Development Framework

4.1.1 ESP32-S3 Microcontroller Architecture

The deployment target is ESP32-S3 system-on-chip (SoC) module manufactured by Espressif Systems [51]. The ESP32-S3 integrates a dual-core Xtensa LX7 processor operating at 240 MHz, providing computational capabilities suitable for embedded deep learning inference while maintaining low power consumption. The Xtensa LX7 cores support single-instruction multiple-data (SIMD) operations through the PIE (Processor Interface Extension) instruction set, enabling efficient execution of vectorized operations common in neural network inference. The ESP-DL inference engine leverages these SIMD capabilities. The SoC includes a 512 KB internal SRAM, 8 MB external PSRAM, and 16 MB external flash memory. The memory architecture is critical for model storage and inference execution:

- **Internal SRAM (512 KB):** High-speed on-chip memory with single-cycle access latency, partitioned between instruction RAM (IRAM), data RAM (DRAM), and DMA-accessible regions. The ESP-DL runtime uses SRAM for frequently accessed weight tiles via a least-recently-used (LRU) caching policy, storing up to 64 KB of active parameters.
- **External PSRAM (8 MB):** Octal SPI PSRAM operating at 80 MHz with Hybrid Wrap burst mode and 32-byte burst length. PSRAM provides the primary storage for model, weights and activations buffer, camera frame buffers, and inference intermediate tensors. The ESP-DL runtime manages PSRAM allocations through the



ESP-IDF heap allocator, with explicit control over memory regions via capability flags.

- **External Flash (16 MB):** QIO-mode (Quad Input/Output) flash storage at 80 MHz clock speed, used for firmware binary storage including model weights embedded in the `.rodata` section. The flash partition table allocates a 4 MB factory application partition. The ESP-DL runtime accesses model weights directly from flash via memory-mapped I/O, caching active tiles in SRAM to mitigate flash access latency.

In this deployment, I used Freenove ESP32-S3-WROOM 1 development board [74] with OV3660 camera sensor connected via 8-bit parallel DVP interface. The OV3660 captures at VGA resolution (640×480) with on-chip JPEG compression, supporting clock frequencies up to 40 MHz [51]. Camera data is transferred to PSRAM through DMA channels. The ESP32-S3's integrated Wi-Fi and Bluetooth capabilities are not utilized in this deployment.

4.1.2 ESP-IDF Development Framework

The firmware developed using Espressif IoT Development Framework (ESP-IDF version 5.3.3), Espressif's official C/C++ development platform for ESP32-series microcontrollers. ESP-IDF provides a FreeRTOS-based real-time operating system with hardware abstraction layers for peripheral management, memory allocation, and power management. The build system uses CMake with Ninja, compiling the application into a single monolithic binary that includes the bootloader, partition table, and application code. The ESP-IDF framework offers the following features relevant to this deployment:

- **Heap allocator:** Manages both internal SRAM and external PSRAM through a unified `heap_caps` API. Memory allocations can be directed to specific memory regions via capability flags (e.g., `MALLOC_CAP_SPIRAM` for PSRAM allocation, `MALLOC_CAP_DMA` for DMA-accessible SRAM).
- **Camera driver:** Configures the DVP camera interface, manages DMA transfers, and provides double-buffered frame capture. The driver supports multiple pixel formats including JPEG, RGB565, and YUV422.
- **SPI flash driver:** provides low-level access to external flash memory, including support for memory-mapped I/O. The `esp_partition` API allows defining custom flash partitions for model storage, while the `esp_spi_flash` API enables direct reads from flash with optional caching.
- **Power management:** ESP-IDF's power management API allows configuring sleep modes and dynamic frequency scaling to optimize energy consumption during idle periods. This enables the duty-cycled operation of inference pipeline, to extend battery life.

4.1.3 ESP-DL Inference Engine

ESP-DL [75] is Espressif's neural network inference library optimized for ESP32-series microcontrollers. The library provides a C++ inference runtime that executes quan-



tized INT8 models directly on the Xtensa LX7 cores, leveraging SIMD instructions for vectorized computation. ESP-DL supports a range of layer types commonly used in convolutional neural networks, including convolution, depthwise convolution, fully connected, activation functions, pooling, and softmax. The library includes optimized kernels for the ESP32-S3's architecture, with specific attention to memory access patterns and cache utilization. The ESP-DL runtime manages model loading, memory allocation for weights and activations, and execution scheduling. Key features relevant to this deployment include:

- **Model loading:** (`dl::Model`) class that loads quantized models from flash RO-DATA, with an optional `minimize()` method that deletes unused variables to reduce memory footprint at the cost of debug capability.
- **Image preprocessing:** (`dl::image::ImagePreprocessor`) class that handles input image transformations required by the models, including resizing, normalization, and color space conversion. The preprocessor supports various pixel formats; for this deployment, input images are processed in RGB888 format for hardware compatibility model input requirements.
- **Detection post-processing:** (`dl::detect::ESPDetPostProcessor`) class that uses the raw output tensors from the detection model to produce final bounding box predictions. The post-processor implements non-maximum suppression (NMS), confidence thresholding, and coordinate transformation for object detection models, handling INT8 dequantization internally.
- **Memory management:** The runtime allocates activation buffers in PSRAM and manages an LRU cache of weight tiles in SRAM to optimize inference performance while respecting memory constraints. The library provides APIs for explicit control over memory allocation and deallocation, enabling the zero-copy buffer reuse strategy described in Section 4.3.3.
- **Software JPEG decoder:** (`dl::image::sw_decode_jpeg`) function that decodes JPEG-compressed camera frames to RGB888 format, as the ESP32-S3 lacks a dedicated hardware JPEG decoder.

4.2 Model Quantization and Format Conversion

Deploying neural network models on microcontroller devices requires converting floating-point to INT8 representations compatible with target inference engine. This section describes the quantization pipeline that conversion of both models ESPDet-Pico and MobileNetV2 from PyTorch FP32 to the ESPDL INT8 format. The conversion pipeline follows the model quantization techniques in Espressif's official ESP-DL deployment documentation [76], adapted to the specific requirements of each model architecture.

4.2.1 The ESP-PPQ Quantization Framework

Both models quantized using ESP-PPQ [76], Espressif's Post-Processing Quantization framework. ESP-PPQ is extended over the open-source PPQ library with hardware-specific quantization configurations targeting ESP-DL inference runtime. The framework

applies per-tensor symmetric quantization with ($s = 2^e$) factors, where floating point values mapped to the 8-bit integer range $[-128, 127]$:

$$x_{\text{int8}} = \text{clamp} \left(\text{round} \left(\frac{x_{\text{float}}}{s} \right), -128, 127 \right) \quad (4.1)$$

where x_{float} is the original floating point value, s is the scale factor, and clamp to ensure the quantized value fits within the 8-bit signed integer range. The quantization exponent e computed during calibration to minimize the quantization error over its representative dataset. The power-of-2 constraint on the scale factor enables efficient dequantization via bit-shift operation on the Xtensa LX7 processor, avoiding costly floating-point operation during inference.

The ESP specific quantization setting `QuantizationSettingFactory.espd1_setting` with the `ESPD1_INT8` target applies a layerwise equalization to distribute weights and activations across consecutive layers, reducing the dynamic range that must be with in 8-bit precision. This strategy particularly important for convolutional layers with highly skewed weight distributions. Thus, this addresses the challenge of MobileNetV2's depthwise separable convolutions, where weight distribution varies significantly across channels.

The ESP-PPQ framework performs quantization first, it collects activation statistics from the calibration dataset to determine optimal scale factors s for each layer. Then, it applies the quantization conversion weights and activations, converting the model to INT8 format. Finally, the quantized model exported in a `.espdl` binary format compatible with the ESP-DL runtime.

4.2.2 The ESPDet-Pico Model Quantization Pipeline

After training the ESPDet-Pico leaf localization model in section 3.3, its PyTorch weights were exported into ONNX (Open Neural Network Exchange) format to obtain a standardized intermediate representation. The export used a fixed input shape of $[1, 3, 320, 416]$ (NCHW) and opset 13 to ensure compatibility with the ESP-DL pipeline. The ONNX model was simplified using `onnxsim` to eliminate redundant operations and fuse batch normalization layers, resulting in a more efficient graph for quantization. Then the ESP-PPQ quantization was applied to the simplified ONNX model with 32 batches of calibration data and equalization enabled. This achieved 73% reduction in model size. Finally, the quantized model was converted to `.espdl` format for deployment on the ESP32-S3, enabling real-time leaf localization. In Figure 4.1 illustrated the complete quantization pipeline.

The Simplified ONNX Format

Pretrained PyTorch ESPDet-Pico model converted to ONNX format using a custom export path that produces six separate output tensors rather than single concatenated tensor used in standard YOLO inference. A custom `ESP_YOLO` wrapper class replaces the standard detection head's forward method with `export_onnx_forward`, which outputs separate box regression and classification score tensors for each of the three detection scales:

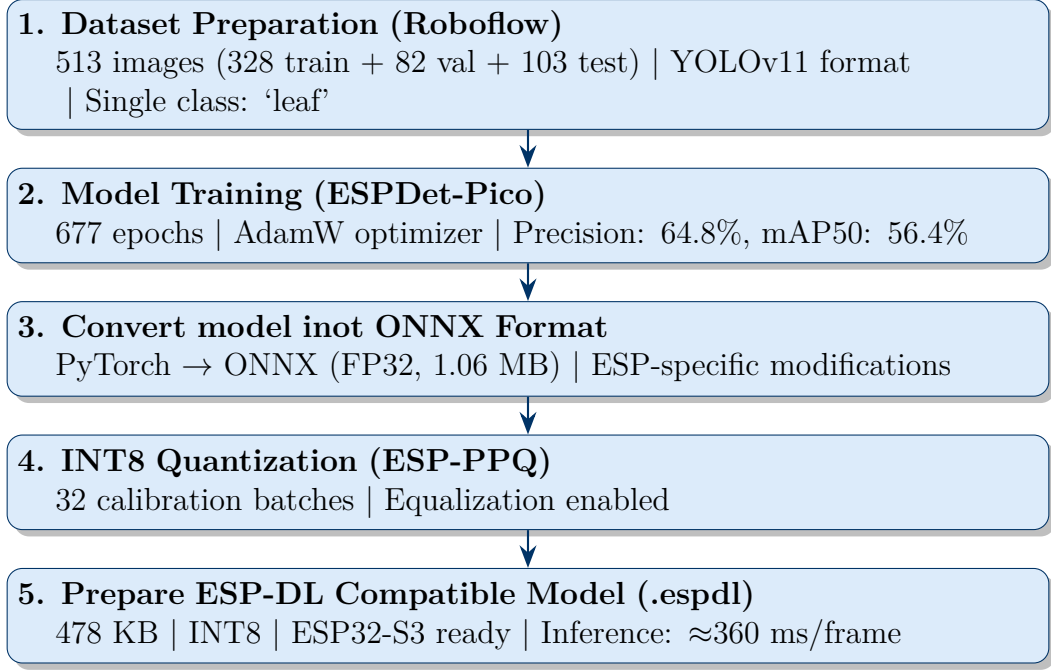


Figure 4.1: ESPDet-Pico quantization pipeline from dataset preparation to ESP32-S3 deployment-ready INT8 model.

$$\text{Output} = \{(\mathbf{B}_s, \mathbf{S}_s)\}_{s \in \{P3, P4, P5\}} \quad (4.2)$$

where $\mathbf{B}_s \in \mathbb{R}^{1 \times 4 \times H_s \times W_s}$ contains raw box edge distances and $\mathbf{S}_s \in \mathbb{R}^{1 \times 1 \times H_s \times W_s}$ contains class logits at scale s . This six-tensor format avoids complex tensor concatenation and reshaping operations that are computationally expensive on the ESP32-S3 and allows the ESP-DL post-processor to handle each scale independently.

INT8 Quantization

The simplified ONNX model is quantized following the procedures of ESP-DL model quantization guide [76]. It needs a calibration dataset to collect activation statistics to determine quantization parameters. The critical constraint for the ESPDet-Pico model was its rectangular input shape (320×416), which requires a custom calibration pipeline. And the batch size must be set to 1 during calibration and proceeds for 32 steps with equalization enabled.

The quantization and format conversion can be performed by calling a single function from the ESP-PPQ module, which internally performs INT8 calibration and exports the result directly to the `.espdl` binary format. Then detection model's INT8 weights become total 478 KB, a 55% reduction from the 1.06 MB FP32 model.

4.2.3 The MobileNetV2 Model Quantization Pipeline

To deploy the MobileNetV2 disease classification model on the ESP32-S3, the trained network converted into ONNX format and then quantized to **INT8** using **ESP-PPQ**. I used similar quantization configuration as the ESPDet-Pico model, ensuring ESP-DL compatibility. The same steps as ESPDet-Pico, with adjustments for the model's input

shape (128×128) and architecture. The complete quantization pipeline is illustrated in Figure 4.2.

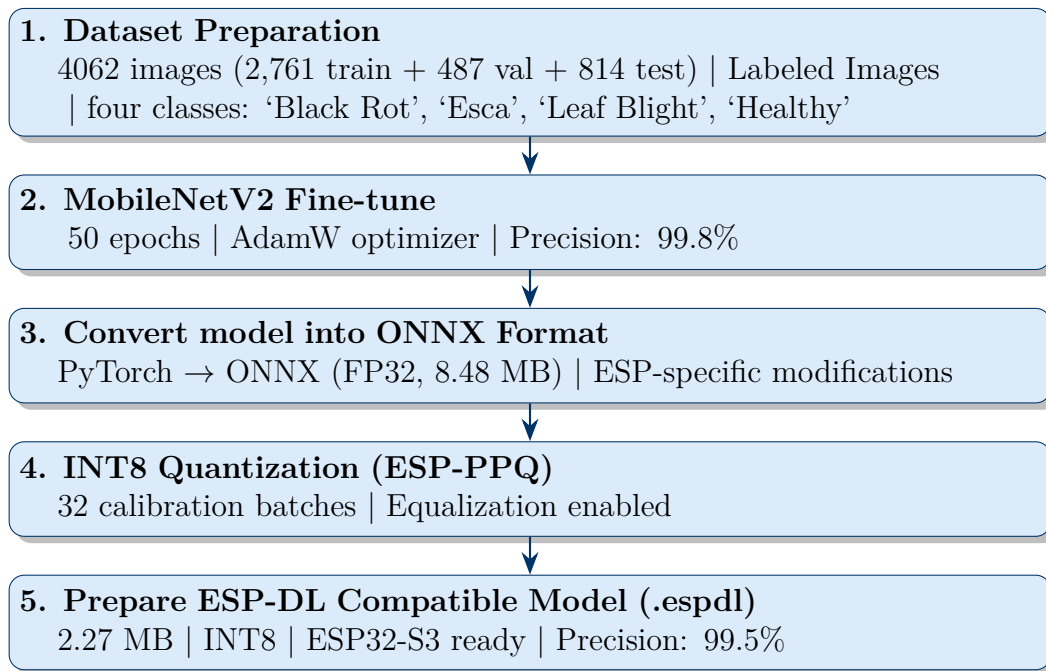


Figure 4.2: MobileNetV2 quantization pipeline from dataset preparation to ESP32-S3 deployment-ready INT8 model

4.2.4 Quantization Accuracy Preservation

Post-training quantization introduces rounding errors that can degrade model accuracy. The layerwise equalization strategy is selected specifically to address this challenge. To contextualize the quantization accuracy trade-off, Espressif’s quantization documentation [76] reports comparative results for three strategies applied to a standard MobileNetV2 model (ImageNet classification, 1000 classes):

- **Default 8-bit quantization:** Produces 25.6% last-layer error, reducing Top-1 accuracy from 71.9% (float) to 60.5%.
- **Mixed precision (INT16 for worst layer):** Reduces last-layer error to 9.1%, achieving 69.6% accuracy.
- **Layerwise equalization:** Achieves 9.0% last-layer error with 69.8% accuracy, matching mixed precision while maintaining uniform INT8 quantization.

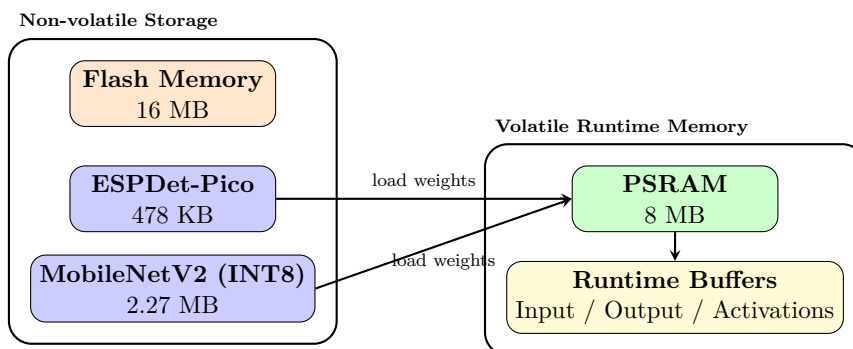
The equalization approach is adopted for both models because it preserves uniform INT8 quantization while achieving accuracy comparable to more complex strategies. The grape disease MobileNetV2 model maintains test accuracy above 99.5% after INT8 quantization, with less than 0.3 percentage point degradation from the FP32 baseline. This robustness reflects MobileNetV2’s batch normalization layers and the inherently quantization-friendly nature of inverted residual blocks [73].

4.3 Firmware Architecture and Memory Management

Deploying two neural network models on a microcontroller with 8 MB PSRAM, 16 MB flash RAM, and 512 KB SRAM imposes careful memory management requirements. This section presents the firmware architecture decisions governing model storage, initialization sequencing, and buffer allocation strategies, each of which critically influences both deployment reliability and inference performance.

4.3.1 Model Storage and Loading Strategy

A key design decision in the firmware architecture is the strategy for storing and loading model weights into the inference engine. Following training, INT8 quantization, conversion to `.espd1` format, the neural network binaries are stored in the flash memory of the ESP32-S3, which is non-volatile and has a capacity of 16 MB. The firmware is designed to load these models into PSRAM at runtime for inference execution as shown in Figure 4.3.



- Both models packed into a single binary (`grape_leaf_detect.espd1`, 2.73 MB total) stored in flash memory
- PSRAM reserved for runtime tensors, activation buffers, and camera framebuffers

Figure 4.3: Model storage and loading strategy for ESP32-S3 deployment. Both ESPDet-Pico and MobileNetV2 models are embedded in flash memory and loaded into PSRAM at runtime for inference execution.

Model Storage Strategies

The strategy for loading model weights into the inference engine significantly affects initialization latency, runtime reliability, and PSRAM availability. Three distinct approaches were evaluated during firmware development:

1. Dedicated Flash Partition Loading: Models stored in custom flash partitions at a fixed offset and loaded directly from the partition. This approach introduces cache coherency issues and memory management unit (MMU) mapping conflicts when accessing partition data, resulting in non-deterministic model loading failures that preclude reliable autonomous deployment.

2. Memory-Mapped Access: Direct memory-mapped I/O with, reading model parameters directly from flash without copying into PSRAM. Although this approach min-

imizes memory footprint, it exhibits instability under high memory utilization and cache lines holding model parameters are thrown out when the camera DMA controller claims buffer regions, causing inference failures.

3. RODATA Embedding with Direct Flash Access: Both models compiled into the firmware binary as constant arrays in the `.rodata` section. During the build process, the ESP-IDF build system uses the CMake function to convert the `.espd1` model files into object files linked directly into the final firmware image. The GCC linker automatically generates special symbols marking the memory addresses where each embedded binary begins and ends. Models remain in flash and are accessed directly through the instruction cache without copying to PSRAM. The ESP-DL runtime internally caches frequently accessed weight tiles in SRAM via an LRU eviction policy (up to 64 KB), balancing flash access latency with memory capacity.

The RODATA approach reduces initialization time for both models combined, while conserving PSRAM exclusively for activation buffers and camera framebuffers. Hence, this strategy was adopted for the production firmware, ensuring stable model loading and maximizing available PSRAM for runtime operations.

4.3.2 Initialization Order of Camera and Models

One of the most challenging issues faced during firmware deployment was the initialization order between the camera module and the neural network models. The ESP32-S3's PSRAM is shared between the camera frame buffers and the model activation buffers, and its allocation strategy can lead to fragmentation that prevents successful model loading if not managed correctly. If the camera is initialized first, it allocates large contiguous buffers for frame capture, which can fragment the PSRAM heap and block subsequent allocations for model weights and activations. Conversely, if the models are initialized first, they can secure their required contiguous memory regions before the camera's DMA buffers are allocated, allowing the system to function correctly despite fragmentation caused by the camera. This critical dependency was identified through systematic testing of different initialization sequences and monitoring PSRAM usage at each step. The final adopted sequence is as follows, illustrated in Figure 4.4:

4.3.3 Buffer Allocation and Reuse Strategy

To minimize runtime memory allocation overhead and prevent heap fragmentation during continuous operation, the inference pipeline employs a **single-allocation, zero-copy buffer strategy**. This approach involves pre-allocating all necessary buffers for input images, intermediate activations, and output predictions in PSRAM during the initialization phase, and reusing these buffers across inference iterations without dynamic allocation or deallocation. The key buffers allocated are as follows:

- **Classification input buffer (49 KB):** A single $128 \times 128 \times 3$ RGB888 buffer allocated in PSRAM during classifier initialization and reused across all detected leaf regions within a frame. This eliminates repeated memory allocation/free calls that would accumulate heap fragmentation over hours of operation.
- **Camera framebuffers (~128 KB):** Two framebuffers allocated by the camera



Figure 4.4: Initialization sequence and PSRAM utilization for model loading and camera initialization.

driver in PSRAM via `CAMERA_FB_IN_PSRAM`. The driver manages double-buffering internally: one buffer captures the next frame while the other undergoes processing.

- **JPEG decode buffer (~900 KB):** The decoded RGB888 frame ($640 \times 480 \times 3 = 921,600$ bytes) is allocated dynamically by the software JPEG decoder. This temporary allocation is freed immediately after detection inference completes and leaf regions are extracted from the decoded buffer.
- **Model activation buffers (~2 MB total):** Intermediate feature maps allocated by the ESP-DL memory manager during model initialization. These buffers persist throughout execution using a static allocation plan computed offline during quantization.

This buffer management strategy ensures that the PSRAM is utilized efficiently without fragmentation, enabling stable long-term operation of the inference pipeline while maintaining real-time performance.

4.3.4 Memory Utilization Summary

In the ESP32-S3-WROOM-1 development board 8 MB PSRAM and 16 MB Flash RAM are available for model storage and runtime operations. In table 4.1 summarizes the memory utilization for the deployed firmware, including model sizes, buffer allocations, and remaining free memory after initialization. After all allocations, approximately 3.9 MB of PSRAM remains free for runtime operations, providing stable room for future feature additions.

Component	Size	Location	Usage
Bootloader	23 KB	Flash	Boot sequence
Partition Table	3 KB	Flash	Memory map
Application Code	4.4 MB	Flash	Firmware
ESPDet-Pico	478 KB	Flash (packed)	Leaf detection
MobileNetV2	2.27 MB	Flash (packed)	Disease classification
DMA Buffers (16 nodes)	16 KB	SRAM	Hardware staging
JPEG Frame Buffers (2×)	120 KB	PSRAM	Double buffering
RGB888 Decoded	921 KB	PSRAM	$640 \times 480 \times 3$
Detection Input	492 KB	PSRAM	$416 \times 320 \times 3$
Classification Input	49 KB	PSRAM	$128 \times 128 \times 3$
Model Activations	2.6 MB	PSRAM	Runtime tensors
Total Used PSRAM	4.1 MB	PSRAM	Active buffers
Free PSRAM	3.9 MB	PSRAM	Available

Table 4.1: Memory utilization summary for ESP32-S3 deployment firmware

4.4 Inference Pipeline Implementation

The complete inference pipeline starts with camera acquisition of JPEG-compressed frames, followed by software decoding to RGB888 format. The decoded image is then processed by the ESPDet-Pico model to detect leaf bounding boxes. Each detected leaf region is cropped and resized to 128×128 for input into the MobileNetV2 classifier, which outputs disease predictions. Finally, predictions from all detected leaves are aggregated to produce a frame-level diagnosis. Figure 4.5 illustrates the complete data flow.

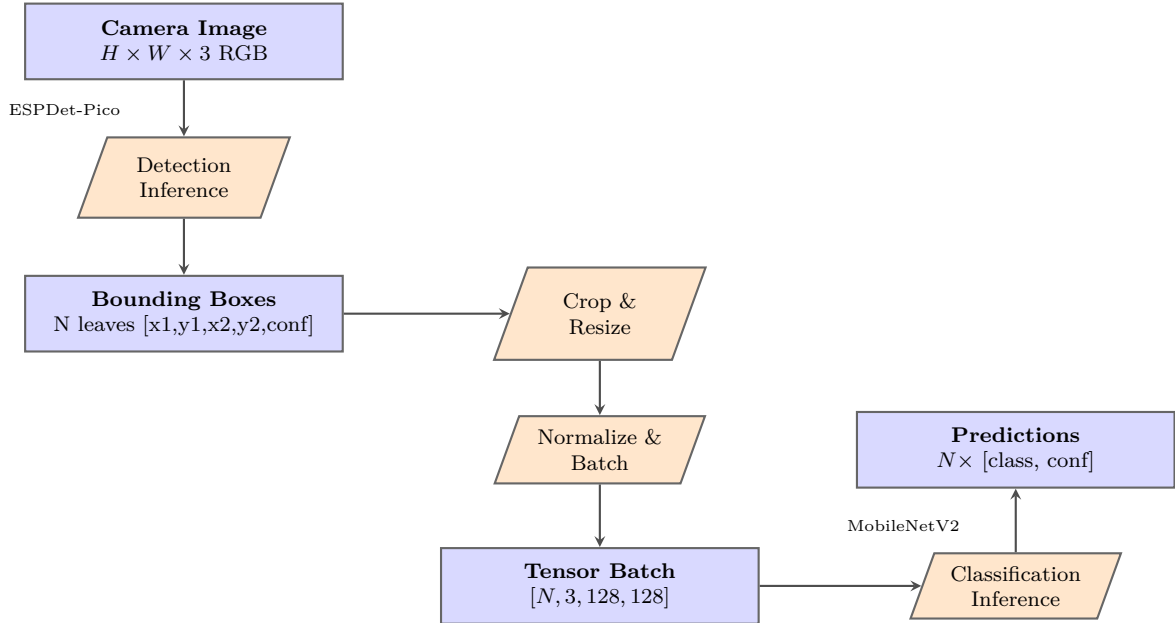


Figure 4.5: End-to-end inference pipeline showing detection, cropping, and classification stages for grape leaf disease detection

4.4.1 Camera Acquisition and JPEG Decoding

The inference pipeline begins with image acquisition using the OV3660 camera module, which is configured to capture frames at VGA resolution (640×480) with JPEG compression. The OV3660's hardware JPEG encoder produces compressed frames that are significantly smaller than raw RGB data, reducing the bandwidth and latency of camera capture. However, the ESP32-S3 lacks a hardware JPEG decoder, necessitating software decoding. The ESP-DL library provides a software JPEG decoder function, which takes the compressed JPEG data and outputs a decoded RGB888 image.

4.4.2 Leaf Localization and Coordinate Mapping

The decoded RGB888 frame undergoes ESPDet-Pico detection inference to localize individual leaf regions. The imagePreprocessor function from ESP-IDF resizes the 640×480 input to 416×320 via letterbox padding with gray fill (RGB= [114, 114, 114]), preserving the original aspect ratio.

Detection inference produces six output tensors (box and score predictions at three scales), processed with the following parameters:

- **Confidence threshold:** $\tau \geq 0.25$, tuned empirically to balance recall and computational cost.
- **NMS IoU threshold:** 0.7, eliminating duplicate detections of overlapping leaves.
- **Anchor strides:** {8, 16, 32} for the three detection scales, with grid sizes corresponding to P3/8, P4/16, and P5/32 feature maps.

Detected bounding box coordinates are mapped from the 416×320 detection space back to the original 640×480 frame space via inverse letterbox transformation:

$$x_{\text{orig}} = \frac{(x_{\text{det}} - \text{pad}_x) \times W_{\text{orig}}}{W_{\text{det}} - 2\text{pad}_x}, \quad y_{\text{orig}} = \frac{(y_{\text{det}} - \text{pad}_y) \times H_{\text{orig}}}{H_{\text{det}} - 2\text{pad}_y} \quad (4.3)$$

This transformation ensures that cropped leaf regions align precisely with the original RGB888 frame for downstream classification.

4.4.3 Leaf Region Cropping and Preprocessing

For each detected bounding box (x_1, y_1, x_2, y_2) returned by ESPDet-Pico, the corresponding leaf region is cropped from the decoded 640×480 RGB888 frame and resized to 128×128 pixels for MobileNetV2 classification. The firmware implements this as a custom `crop_and_resize` function that performs cropping, square expansion, and resizing in a single pass, writing the result directly into the pre-allocated PSRAM classification buffer without any intermediate allocation.

The function first clamps the bounding box coordinates to the frame boundaries. It then computes the box center (c_x, c_y) and expands the rectangular region to a square with side

length $L = \max(x_2 - x_1, y_2 - y_1)$, centered on the same midpoint:

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2}, \quad x'_1 = c_x - \frac{L}{2}, \quad y'_1 = c_y - \frac{L}{2} \quad (4.4)$$

This square expansion ensures that the leaf is not stretched or compressed when resized to the square 128×128 classifier input, preserving the original aspect ratio. The expanded square region is then sampled into the 128×128 output buffer using nearest-neighbor interpolation. For each output pixel (x, y) , the source pixel coordinates in the original frame are:

$$\text{src}_x = x'_1 + \left\lfloor \frac{x \cdot L}{128} \right\rfloor, \quad \text{src}_y = y'_1 + \left\lfloor \frac{y \cdot L}{128} \right\rfloor \quad (4.5)$$

If the source pixel lies within the frame ($0 \leq \text{src}_x < 640$ and $0 \leq \text{src}_y < 480$), its RGB values are copied directly. Otherwise, if the square expansion extends beyond the image edges for leaves near the frame boundary, the pixel is filled with gray padding (RGB=[127, 127, 127]) to maintain a consistent input size. The final pixel value assignment is thus:

$$I_{128}[y, x] = \begin{cases} I_{\text{frame}}[\text{src}_y, \text{src}_x] & \text{if } 0 \leq \text{src}_x < W \text{ and } 0 \leq \text{src}_y < H \\ (127, 127, 127) & \text{otherwise (gray padding)} \end{cases} \quad (4.6)$$

Figure 4.6 illustrates the cropping and resizing process for a detected leaf region. The original bounding box (red dashed) is expanded to a square (blue) centered on the same point, then resized to 128×128 for classification input. Gray padding is applied if the square extends beyond frame boundaries.

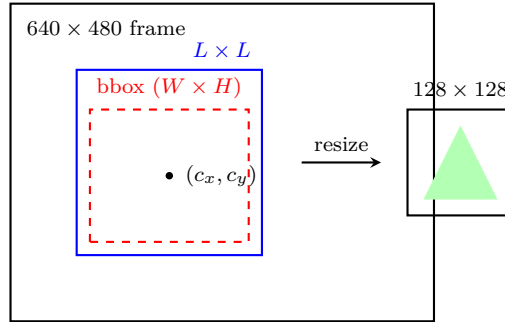


Figure 4.6: The cropping and resizing process of the detected leaf region

The output is written directly into the persistent $128 \times 128 \times 3$ classification input buffer in PSRAM. This buffer is allocated once during initialization and reused for every detected leaf, as part of the buffer allocation and reuse strategy described in Section 4.3.3.

4.4.4 Disease Classification and INT8 Dequantization

The cropped leaf image undergoes MobileNetV2 classification inference. The model outputs a 4-dimensional INT8 logit vector representing class scores for **Black_rot**, **Esca**, **Healthy**, and **Leaf_blight**. The INT8 logits are dequantized to floating-point values. Then, the dequantized logits are passed through a softmax function to obtain class probabilities. The predicted class is selected as the one with the highest probability, along with its confidence score.

$$\text{logit}_{\text{float}}^{(c)} = \text{logit}_{\text{int8}}^{(c)} \times 2^e \quad (4.7)$$

where e is the quantization exponent embedded in the output tensor’s metadata. Unlike the detection model, whose post-processor handles dequantization internally, the classification model outputs raw INT8 logits that require explicit dequantization before applying softmax. The dequantized logits are then passed through a softmax function to obtain class probabilities, and the predicted class is selected as $\arg \max_c p^{(c)}$ along with its confidence score.

4.4.5 Multi-Leaf Aggregation Strategies

Since the grapevine images typically contain multiple leaves (>15 visible leaves per frame), the final frame-level disease diagnosis cannot rely on a single leaf prediction or transmit the prediction results of all leaves in not suitable for LoRaWAN communication. Instead, it must aggregate predictions across all detected leaves to produce a robust diagnosis for the entire frame and get a single prediction result for LoRaWAN transmission.

The aggregation module implements three strategies, drawing from multi-instance learning (MIL) principles [77]:

1. Detection-Confidence Weighting (Baseline): Each leaf’s class probabilities are weighted by its detection confidence c_{det} :

$$p_{\text{frame}}^{(c)} = \frac{\sum_{i=1}^N c_{\text{det}}^{(i)} \cdot p_i^{(c)}}{\sum_{i=1}^N c_{\text{det}}^{(i)}} \quad (4.8)$$

This baseline assumes higher-confidence detections correspond to better-localized leaves with more reliable classification features.

2. Entropy-Based Uncertainty Weighting: Shannon entropy [78] quantifies per-leaf classification uncertainty:

$$H_i = - \sum_{c=1}^C p_i^{(c)} \log p_i^{(c)} \quad (4.9)$$

Leaves with low entropy (confident predictions) receive higher aggregation weight than ambiguous predictions. The normalized weight becomes:

$$w_i = c_{\text{det}}^{(i)} \cdot \left(1 - \frac{H_i}{\log C} \right) \quad (4.10)$$

where $\log C$ normalizes entropy to $[0, 1]$. This approach downweights leaves near class boundaries (e.g., Black Rot OR Esca confusion) exhibiting high classification uncertainty.

3. Hybrid Spatial-Quality Weighting: Bounding box geometry provides additional quality indicators, combined with detection confidence and entropy:

- **Relative size:** $r_{\text{size}} = A_{\text{bbox}}/A_{\text{frame}}$, penalizing excessively small (noise) or large (misdetections) boxes outside the range $[0.005, 0.5]$ of frame area.

- **Aspect ratio:** Scoring function centered at ideal ratio 1.0 with tolerance 0.5, penalizing elongated boxes corresponding to stems or trellis wires.
- **Centrality:** $d_{\text{center}} = \|(x_c, y_c) - (W/2, H/2)\|$, downweighting edge-positioned leaves that may be partially occluded or distorted by lens aberration.

The combined quality score $q_i \in [0, 1]$ multiplies the detection-entropy weight:

$$w_i^{\text{hybrid}} = c_{\text{det}}^{(i)} \cdot \left(1 - \frac{H_i}{\log C}\right) \cdot q_i \quad (4.11)$$

The frame-level diagnosis selects the class $c^* = \arg \max_c p_{\text{frame}}^{(c)}$ with the aggregated confidence as the final output.

4.5 Implementation Constraints and Design Trade-offs

Developing and deploying an Edge AI system on the ESP32-S3 microcontroller imposed high resource constraints that required careful design trade-offs. Some from the most significant constraints were the limited 8 MB PSRAM, lack of floating-point SIMD acceleration, low LoRaWAN bandwidth, and power limitations of battery operation.

One from the most significant constraints was the PSRAM limit. Even though higher input resolutions improve detection accuracy for small or distant leaves, they increase peak memory usage quadratically ($\mathcal{O}(N^2)$). For this reason, the system restricted its input to 416×320 . While a higher resolution (e.g., 800×600) would improve detection of smaller leaves at a distance, it would exceed the PSRAM capacity when combined with the classification model buffers. Hence, This resolution was selected as a trade-off optimal point, maximizing detection performance while maintaining some free memory space to prevent from crashes during operation.

Another critical constraint is the lack of floating point SIMD acceleration on the ESP32-S3's Xtensa LX7 cores. Therefore, the system adopted INT8 quantization via the ESP-PPQ framework (Section 4.2), reducing model size and leveraging the ESP32-S3's vector instructions for integer arithmetic. The trade-off was a minor degradation in classification accuracy, which is operationally acceptable compared to the non-viability of FP32 deployment on this hardware.

Additionally, the communication constraint of LoRaWAN network, It is impractical to transmit raw images or even intermediate feature vectors over LoRaWAN due to its limited payload size. Therefore, the system performs complete inference and aggregation on-device, transmitting only the final diagnosis. This design choice sacrifices the ability of detection but enables deployment in remote vineyard locations, where LoRaWAN is the only viable communication option.

The final and most critically, the battery power constraint required careful management of the inference duty cycle. This duty-cycled operation is essential for achieving months of operation on a single battery charge.

Design Property	Constraint	Adopted Trade-off
Model Precision	No FP32 SIMD acceleration	INT8 Quantization
Input Resolution	8 MB PSRAM limit	416×320 detection input
Communication	Low LoRaWAN bandwidth	Edge aggregation (local decisions)
Power	Battery	Duty-cycled sleep architecture

Table 4.2: Implementation constraints and design trade-offs of deploying Edge AI system on ESP32-S3

Having described the complete deployment pipeline, the implementation constraints, and the design trade-offs encountered during firmware development, the next chapter evaluates the system’s performance in terms of inference latency, classification accuracy, and memory utilization.

Chapter 5

Performance Evaluation and Discussion

Following the system implementation described in Chapter 4, this chapter discusses the performance evaluation of the implemented Edge AI system for the grape leaf disease detection system by deploying it on the ESP32-S3 board. The objective is to evaluate and validate that the proposed system meets the operational requirements in terms of speed, precision, and stability.

The experimental analysis begins in Section 5.1 by introducing the experimental setup. Section 5.2 presents the detailed experimental inference cycle, verifying the functional correctness of the detection, classification, and aggregation logic. The subsequent analysis focuses on the system's computational performance. Section 5.3 by providing detailed latency analysis of each inference pipeline stage. Section 5.4 examines memory footprint and runtime stability. The chapter concludes in Section 5.6 with a discussion of some trade-offs and the current system limitations.

All measurements reported in this chapter are obtained from the experimental setup of firmware running on the ESP32-S3-WROOM-1 development board with OV3660 camera, using serial monitor output at 115,200 baud for timing and memory log data.

5.1 Experimental setup

5.1.1 Hardware and Firmware Configuration

The deployable firmware was compiled using the ESP-IDF development framework and flashed into the Freenove ESP32-S3-WROOM-1 development board. The device was powered via the USB, 5 V regulated to 3.3 V. The USB connection also served as a data channel for serial monitoring. The firmware was configured to run the complete inference pipeline. Additionally, it is configured with microsecond-resolution timers and heap integrity checks, painting data to the serial monitor at 115,200 baud.

The OV3660 camera module was connected to the board using a ribbon connector and configured for JPEG output at VGA resolution (640×480).

5.1.2 Test Environment

At this time, in the winter season, grapevines are without leaves. Therefore, to simulate field capture conditions in a streaming manner, a printed grapevine banner depicting various leaf densities was used as the test subject. The camera was positioned manually to vary distance and angle, approximating the variability in the field. In this experiment the following parameters were applied to the detection algorithm:

- **Detection Threshold:** $\tau = 0.25$, allowing the system to capture lower-confidence leaf regions.
- **Maximum Leaf Count:** Limited to 10 leaves per frame

5.2 Experimental Inference Results

On the experimental setup described in Section 5.1, experiments were conducted to validate the complete inference pipeline. A complete inference log per cycle was captured via the serial monitor output.

The firmware successfully initialized the OV3660 sensor and the neural network models, verifying the memory mapping strategy described in 4. Upon triggering the inference loop, the camera captures a 640×480 frame and undergoes through the ESPDet-Pico detector. The detector identified 10 regions of interest (ROI). Each leaf was subsequently processed by the MobileNetV2 classifier. Table 5.1 summarizes the raw prediction data for the top detected instances.

Table 5.1: Serial output from a single inference cycle

Leaf ID	Confidence	Primary Class	Prob.	Entropy	Weight (w_i)
0	0.500	Leaf Blight	47.3%	0.895	0.0082
1	0.500	Healthy	69.6%	0.619	0.0925
2	0.321	Healthy	48.6%	0.829	0.0004
3	0.321	Healthy	65.2%	0.728	0.0054
4	0.321	Healthy	69.2%	0.658	0.0217
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
9	0.269	Leaf Blight	61.0%	0.740	0.0107

Aggregation Logic Verification

The serial monitor logs show the aggregation strategy's importance. Notably, **Leaf 1** (Table 5.1) received the highest aggregation weight ($w_1 = 0.0925$). This is consistent with its low classification entropy (0.619 indicating high certainty) and large spatial dimensions (18,081 px²).

Conversely, **Leaf 2** demonstrates the spatial penalty mechanism. Despite having a valid detection confidence (0.321), it received a negligible weight ($w_2 = 0.0004$). This suppression is due to its poor spatial quality, small size, (1,512 px²) and location at the very edge of the frame ($y = 0$), identifying it as likely background noise. Similarly, **Leaf 0** was down-weighted ($w_0 = 0.0082$) due to high entropy (0.895), reflecting an ambiguous prediction.

The final aggregated result was:

- **Diagnosis:** Healthy
- **Aggregated Confidence:** 58.2%
- **Total Weight Mass:** 0.1883

This demonstrates that the firmware correctly down-weights ambiguous or poorly localized predictions, preventing outliers from corrupting the frame-level diagnosis.

5.3 Inference Latency Analysis

Per-Stage Latency Breakdown

The inference pipeline has been configured with microsecond timing records at each processing stage. Table 5.2 presents the measured latency for each stage during every inference cycle capturing grapevine image (10 detected leaves for experiment). The breakdown includes camera capture, JPEG decoding, leaf detection (ESPDet-Pico), and disease classification (MobileNetV2) for each detected leaf. The classification time encompasses both the crop and resize preprocessing as well as the INT8 forward pass on the Xtensa LX7.

Table 5.2: Per-stage latency breakdown for the inference pipeline (10 detected leaves)

Pipeline Stage	Latency (ms)	Percentage
Camera capture (JPEG)	78	1.2%
JPEG decode (RGB888)	51	0.8%
Leaf detection	361	5.7%
Disease classification ($\times 10$ leaves,)	5,886	92.3%
Total end-to-end	6,376	100%

The disease classification stage (MobileNetV2) took 5,886 ms for 10 leaves. This stage dominates the overall latency at 92.3%, while leaf detection accounts for 5.7% and image acquisition (capture + decode) contributes only 2.0%. The classification time scales linearly with the number of detected leaves, as each leaf is processed sequentially through the MobileNetV2 model.

The disease classification latency of 5,886 ms for 10 leaves corresponds to approximately 589 ms per leaf. This per-leaf time includes crop and resize preprocessing (~ 1 -2 ms), tensor setup (~ 0.1 ms), the INT8 forward pass (~ 550 ms), and softmax post-processing (~ 1 ms). The forward pass constitutes approximately 93% of the per-leaf classification time and is dominated by depthwise separable convolution operations executing in INT8 arithmetic on the Xtensa LX7 SIMD units.

In contrast, the leaf detection stage processes the full 416×320 input image and completes in 361 ms. This time includes letterbox preprocessing (~ 22 ms), the ESPDet-Pico INT8 forward pass (~ 340 ms), and non-maximum suppression post-processing. The favorable detection speed relative to classification reflects ESPDet-Pico’s substantially smaller parameter count (360K vs ~ 2.2 M for MobileNetV2) and the computational efficiency of the anchor-free single-pass architecture.

In image acquisition stages, camera capture and JPEG decode together contribute only 129 ms (2.0% of total latency), confirming that neural network inference, not I/O or preprocessing, is the computational bottleneck in embedded edge AI deployments.

The latency breakdown reveals that the computational bottleneck in the pipeline is the disease classification stage, which is expected given the complexity of the MobileNetV2 architecture and the use of INT8 quantization on a resource-constrained embedded platform. The detection stage, while processing the full input image, is significantly faster due to the efficiency of the ESPDet-Pico model and its smaller parameter count. Im-

age acquisition stages are negligible in terms of latency, confirming that neural network inference is the primary contributor to end-to-end processing time in this application.

End-to-End Pipeline Latency and Throughput

The end-to-end latency of 6,376 ms for a frame with 10 detected leaves corresponds to a throughput of approximately 0.16 frames per second (FPS). The latency scales linearly with the number of detected leaves due to the sequential application of classification to each detected region.

$$T_{\text{total}} = T_{\text{capture}} + T_{\text{decode}} + T_{\text{detect}} + N \cdot (T_{\text{crop}} + T_{\text{classify}}) + T_{\text{aggregate}} \quad (5.1)$$

where N denotes the number of detected leaves and $(T_{\text{crop}} + T_{\text{classify}}) \approx 589$ ms per leaf (of which the INT8 forward pass accounts for ~ 550 ms). This linear relationship yields the following throughput estimates:

Table 5.3: End-to-end latency and throughput versus number of detected leaves

Number of Leaves	Estimated Latency (ms)	Throughput (FPS)
1	$\sim 1,080$	0.93
3	$\sim 2,260$	0.44
5	$\sim 3,440$	0.29
7	$\sim 4,610$	0.22
10	6,376	0.16

At the maximum detection count of 10 leaves per frame, the system achieves 0.16 FPS, corresponding to one complete inference cycle every 6.4 seconds. For the target application of periodic vineyard monitoring, where disease symptoms evolve over days rather than hours, the system is designed to perform four inference cycles per day at 6-hour intervals. The resulting duty cycle is negligible (6.4 s active out of 21,600 s per cycle $\approx 0.03\%$), enabling extreme energy efficiency, as will be explained in detail in Section 5.5.

5.4 Runtime Memory Utilization

5.4.1 Memory Footprint Analysis

The system’s memory footprint is analyzed in terms of PSRAM usage, flash storage, and internal SRAM. This analysis is based on the log serial output from the firmware, which provides detailed breakdowns of memory allocation at each initialization stage and during the inference cycles.

The total PSRAM consumption is approximately 4,280 KB (52.2% of 8 MB), leaving 3,910 KB free during steady-state inference. This memory budget includes:

- **ESPDet-Pico activation buffers:** $\sim 1,095$ KB for detection model activation tensors and internal buffers.



- **MobileNetV2 activation buffers:** $\sim 2,839$ KB for classification model activation tensors plus the 49 KB persistent input buffer.
- **Camera DMA buffers:** 2×60 KB = 120 KB for double-buffered JPEG frame capture in PSRAM.
- **Driver and runtime overhead:** ~ 177 KB for camera driver allocations, heap metadata, and memory alignment.
- **JPEG decode buffer:** ~ 900 KB (dynamically allocated and freed per frame).

The 3,910 KB free PSRAM during inference provides sufficient margin for the temporary JPEG decode buffer allocation (~ 900 KB) and other runtime overhead, with approximately 3 MB remaining as a safety margin against unexpected allocations or memory alignment overhead.

Another, the firmware binary occupies ~ 4 MB of the 16 MB flash storage. The flash storage is used for the firmware code, model weights, and static data. The internal SRAM (512 KB total) is primarily used for runtime stack and heap allocations.

5.4.2 Long-Term Stability

The single-allocation reuse buffer strategy (Section 4.3.3) enables the system to maintain a stable memory footprint across inference cycles. The system monitors free PSRAM before and after each inference cycle, verifying that the post-inference free memory returns to the same baseline value (3,910 KB) after every cycle.

Except for the JPEG decode buffer, which is allocated and freed within each inference cycle, all other buffers (classification input, model activations, camera framebuffers) are allocated once during initialization and reused across all subsequent inference cycles without reallocation. This design choice eliminates the risk of memory fragmentation and ensures that the system's memory usage remains consistent over time, even with repeated inference cycles. The stability of the free PSRAM readings across multiple inference cycles confirms that there are no memory leaks.

5.5 Power Consumption Analysis

5.5.1 Active and Sleep Power Measurements

Power consumption is a critical metric for the deployed vineyard monitoring system, which is designed for long-term autonomous operation on battery power. The system's energy efficiency is evaluated by characterizing power consumption in both active inference mode and deep sleep mode. In active mode, the camera is active and utilizing 98 mA [79] for image capture and processing in 78 ms, while the ESP32-S3 draws approximately 107.9 mA during the full inference cycle with PSRAM access and peripheral operation. While in deep sleep mode, the system draws only 8 μ A, according to the ESP32-S3-WROOM_1 board datasheet specifications [80]. The following analysis combines these measurements to estimate the average power consumption over a typical duty cycle of four inference cycles per day at 6-hour intervals.

5.5.2 Duty-Cycle Energy Analysis

To accurately estimate the system's energy autonomy, the power consumption profile is decomposed into three distinct operational phases: *Image Capture*, *Inference Processing*, and *Deep Sleep*. The system operates on a 6-hour cycle ($T_{\text{cycle}} = 21,600$ s), performing four diagnoses per day.

Power Source Selection

Given the long sleep intervals, a Lithium Thionyl Chloride (Li-SOCl₂) primary battery was selected as the power source. This offers a nominal voltage of 3.6 V, a capacity of 2400 mAh.

Periodic Operations

The current consumption breakdown for a single cycle is defined as follows:

1. **Image Capture** ($T_1 = 78$ ms): Both the ESP32-S3 SoC and the OV3660 camera are active. The total current draw is the sum of the SoC high-performance mode (107.9 mA) and the camera active current (98 mA), totaling 205.9 mA.
2. **Inference Processing** ($T_2 = 6.22$ s): The camera is powered down immediately after capture to conserve energy. The ESP32-S3 processes the data (detection, cropping, INT8 classification) drawing 107.9 mA.
3. **Deep Sleep** ($T_3 \approx 21,593.7$ s): The system enters deep sleep mode with RTC peripherals active, drawing a quiescent current of 8 μ A (0.008 mA).

Energy Budget Calculation

The total charge consumed per cycle (Q_{cycle}) is calculated by integrating the current over the duration of each phase:

$$Q_{\text{cycle}} = \sum_{i=1}^3 (I_i \cdot T_i) + Q_{\text{others}} \quad (5.2)$$

Substituting the datasheet and measured values:

$$\begin{aligned} Q_{\text{capture}} &= 215.9 \text{ mA} \times 0.078 \text{ s} \approx 16.8 \text{ mAs} \\ Q_{\text{infer}} &= 117.9 \text{ mA} \times 6.298 \text{ s} \approx 742.5 \text{ mAs} \\ Q_{\text{sleep}} &= 0.028 \text{ mA} \times 21,591.6 \text{ s} \approx 604.6 \text{ mAs} \\ Q_{\text{others}} &= (215.9 \times 2.0) + (117.9 \times 0.020) \approx 434.2 \text{ mAs} \end{aligned}$$

The total charge per cycle is $Q_{\text{total}} \approx 1,798.1$ mAs. The weighted average current consumption (I_{avg}) is therefore:

$$I_{\text{avg}} = \frac{1,798.1 \text{ mAs}}{21,600 \text{ s}} \approx \mathbf{0.0832 \text{ mA}} \quad (83.2 \text{ } \mu\text{A}) \quad (5.3)$$

Battery Lifetime Estimation

Based on the battery capacity ($Q_{\text{battery}} = 2,400 \text{ mAh}$), the operational lifetime is estimated as follow:

$$T_{\text{life}} = \frac{Q_{\text{battery}}}{I_{\text{avg}}} = \frac{2,400 \text{ mAh}}{0.0832 \text{ mA}} = \frac{28,846 \text{ hours}}{4} \approx \mathbf{10 \text{ months}}$$

Table 5.4: Detailed energy budget per 6-hour inference cycle

Phase	Duration	Current (mA)	Charge (mAs)	% Energy
Capture (Cam+SoC)	78 ms	215.9	16.8	0.9%
Inference (SoC)	6.298 s	117.9	742.5	41.3%
Deep Sleep (SoC+Cam)	~6.0 h	0.028	604.6	33.6%
Others (Boot/Init + USART)	2.02 s	—	434.2	24.1%
Total			860.0	100%

Table 5.4 summarizes the energy budget per cycle. The inference processing phase accounts for 41.3% of the total charge, while the system overhead (boot/initialization and USART communication) contributes 24.1%. Notably, the camera standby current ($20 \mu\text{A}$) during the extended deep sleep interval constitutes 33.6% of the total energy budget, highlighting the potential benefit of implementing hardware-level power gating (e.g., a MOSFET switch to fully cut camera power during sleep) in a production deployment. With such a power gate reducing the sleep current to $8 \mu\text{A}$ (ESP32-S3 only)

5.6 Discussion

5.6.1 Trade-offs Between Accuracy, Latency, and Power

The design and deployment of the vineyard disease detection system on the ESP32-S3 platform involved several critical trade-offs between accuracy, inference latency, and power consumption. These trade-offs were carefully balanced to meet the constraints of microcontroller-class hardware while achieving sufficient performance for the target application.

Accuracy vs. Model Size: As the model size directly impacts the inference accuracy, the ESPDet-Pico also sacrifices 8.6% relative mAP@50 compared to YOLO11n (56.42% vs 61.75%) in exchange for an 86% parameter reduction that enables in-memory deployment. Another MobileNetV2 at 128×128 input resolution trades its accuracy gains from higher resolution (224×224) for a $3\times$ reduction in activation memory, enabling deployment within the 8 MB PSRAM budget.

Latency vs. Accuracy: The more leaves detected in a captured frame, the higher and more accurate the frame-level diagnosis, but the longer the inference time due to sequential classification. The maximum detection limit of 10 leaves per frame is a design choice that balances the desire for comprehensive leaf-level analysis (which improves accuracy) against the need for timely inference results.

Power vs. Inference Frequency: The duty-cycled design with 6-hour capture intervals allows the system to operate at an average power consumption. However, this low duty cycle means that the system captures disease state only four times per day. For applications requiring more frequent monitoring (e.g., hourly captures during periods of rapid disease progression), the duty cycle can be increased at the cost of proportionally higher energy consumption. Therefore, the system is optimized for periodic monitoring aligned with the temporal dynamics of grapevine diseases, rather than continuous real-time detection.

5.6.2 Limitations and Deployment Considerations

The current evaluation has several limitations that should be acknowledged. Starting from the model’s accuracy, all metrics were obtained from controlled datasets (PlantVillage dataset for classification, Roboflow-annotated images for detection). The system deployment experiment utilized a printed grapevine photo to validate the inference pipeline; while this confirms logical correctness and timing, it does not constitute field-level accuracy validation. On-field testing with live grapevines under natural environmental conditions remains a planned future activity for the upcoming growing season.

Furthermore, disease classification is currently performed on a per-frame basis without temporal tracking across consecutive capture cycles. Implementing change detection or disease progression monitoring would require storing historical predictions in NVS (non-volatile storage) and comparing across frames, which is feasible within the current firmware architecture but not yet implemented.

Additionally, the quantitative evaluation of the aggregation strategy was limited to a single representative frame with 10 detected leaves. A comprehensive comparative evaluation of the three aggregation strategies (baseline, entropy-based, hybrid) requires a large-scale field dataset with ground-truth disease labels for every leaf in a multi-leaf frame, which was unavailable during the laboratory phase.

Despite these limitations, the results presented in this chapter demonstrate a complete functional inference pipeline from camera capture through the result aggregation. It validates the two task-specific neural networks are capable of executing on the ESP32-S3 microcontroller. Moreover, this experiment provided valuable insights into the latency breakdown, memory utilization, and power consumption of the deployed system. The successful execution of the full pipeline on the target hardware confirms the feasibility of microcontroller-based vineyard disease detection and sets the stage for future on-field validation and iterative improvements based on real-world performance data.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis work covered the design and implementation of an Edge AI system for grape leaf disease detection to be deployed on ESP32-S3 microcontroller and integrated into an existing LoRaWAN-based vineyard monitoring system. The Edge AI system provides direct visual-based disease diagnosis at the sensor level. This overcomes the limitations of traditional environmental sensor-based monitoring approaches. The contribution of this work was the development of a complete four-stage inference pipeline, including image acquisition, leaf localization, disease classification, and result aggregation, all executed on the ESP32-S3 microcontroller. By developing this system, the gap between environmental-only monitoring and intelligently disease diagnosis was bridged. Hence, more accurate and timely detection system enabled for grape leaf diseases in vineyards.

The main contribution of this work has been the development of a four-stage inference pipeline executing entirely on ESP32-S3 microcontroller. The includes image acquisition using OV3660 camera VGA resolution (640×480), leaf localization using the ESP-Optimized YOLO11n model (ESPDet-Pico), multi-class disease classification using a lightweight MobileNetV2 model, and weighted result aggregation formatted for LoRaWAN transmission. Performing this complete inference pipeline at the Edge eliminates the need of transmitting raw images over bandwidth-constrained LoRaWAN network.

The leaf localization model was developed from the ESP-Optimized variant of YOLO11n architecture, named ESPDet-Pico. Systematic achitectural pruning has been applied. Including the replacement of standard convolutions with depthwise separable convolutions, compound scaling, and spatial-channel downsampling. These reduced the parameter count by 86.1% (from 2.59M to approximately 360K) and the model size by 79.8% (from 5.23 MB to 1.06 MB in FP32). Despite this aggressive pruning, ESPDet-pico retained almost the same detection performance as the baseline YOLO11n, achieving 5.642% mAP@50 while the yolo11n achieved 61.75% mAP@50. Besides, post-training INT8 quantization via the ESP-PPQ framework further compressed the model to 478 KB, enabling concurrent in-memory deployment alongside the classification model.

The classification model was developed by fine-tuning a MobileNetV2 architecture on the PlantVillage grape leaf dataset. The architecture was pretrained on ImageNet and then fine-tuned on the grape leaf dataset through a two-stage progressive transfer learning strategy. The model achieved a test accuracy of $99.80\% \pm 0.065\%$ across four disease classes (Black Rot, Esca, Healthy, and Leaf Blight). The consistency across four independent training runs ($\sigma = 0.065\%$) confirmed the robustness and reproducibility of the training protocol. Post-training INT8 quantization reduced the model size from 8.48 MB to 2.27 MB with less than 0.3 percentage point accuracy degradation, demonstrating MobileNetV2's suitability for quantization-constrained edge deployment due to its inverted residual architecture and batch normalization layers.

Since all the detected leaves in a frame undergo classification, the number of leaf-level predictions can be large, unsuitable for direct transmission over LoRaWAN. To produce a concise and reliable frame-level diagnosis, three weighted aggregation strategies were implemented. Hybrid spatial-quality weighting, which combines leaf detection confidence, classification entropy, and spatial quality of the detected leaf (relative size, aspect ratio, and centrality), ensures that the final diagnosis reflects reliable predictions. This aggregated result simply transmits over LoRaWAN and can be visualized on the existing Grafana dashboard. The aggregation logic was experimentally verified to effectively down-weight low-quality detections, preventing outliers from corrupting the frame-level diagnosis and ensuring that the final LoRaWAN payload reflects a reliable diagnostic result.

The complete Edge AI system was compiled in C++ using the ESP-IDF framework and deployed on the ESP32-S3-WROOM-1 development board. This board has a dual-core Xtensa LX7 microcontroller running at up to 240 MHz, 512 KB of SRAM, 8 MB of external PSRAM, and 16 MB of flash memory connected to the OV3660 camera module. This undergoes experimental evaluation of the system's inference latency, memory usage, and power consumption under realistic operating conditions. End-to-end inference latency was measured at 6,376 ms for a frame containing 10 detected leaves. The total PSRAM consumption was approximately 4.1 MB (51% of the available 8 MB), with 3.9 MB remaining free during steady-state operation. The single-allocation buffer reuse strategy eliminated memory fragmentation and confirmed long-term runtime stability.

Another important aspect of the system discussed in this thesis is the power consumption analysis. The system's average current consumption was approximately $83.2\ \mu\text{A}$ during a duty-cycled operation of four inference cycles per day at 6-hour intervals, accounting for boot and initialization overhead, camera standby current during deep sleep, USART communication with the STM32 EndNode, and the full inference pipeline. Based on a 2,400 mAh Li-SOCl₂, the estimated operational lifetime is approximately 10 months. However, the camera standby current during deep sleep constitutes a significant portion of the energy budget. Implementing hardware-level power gating to reduce this standby current from $20\ \mu\text{A}$ to $8\ \mu\text{A}$ could extend the estimated battery lifetime.

In summary, this thesis work successfully developed and evaluated a complete Edge AI system for grape leaf disease detection system. This can clearly demonstrate the feasibility of deploying deep learning inference pipelines on microcontroller-class devices for precision agriculture applications. The system's design and optimization strategies effectively balanced the trade-offs between model complexity, inference latency, memory usage, and power consumption, establishing a practical foundation for intelligent vineyard monitoring systems that combine the long-range, low-power reliability of LoRaWAN communication with optimized deep learning models deployed directly at the edge.

6.2 Future Work

While this thesis work showed the feasibility of deploying deep learning models on resource-constrained microcontrollers for grape leaf disease detection, several aspects remain open for future research to enhance the system’s performance, and practical deployment.

The most important next step to be done is a comprehensive on-field deployment with live grapevines under natural environmental conditions. The current evaluation relied on controlled datasets and a printed grapevine image due to the winter season. Field testing under varying environmental conditions is essential to quantify the system’s real-world detection and classification accuracy. Moreover, field validation would provide the large-scale, per-leaf annotated dataset required for rigorous comparative evaluation of the three implemented aggregation strategies to determine the optimal strategy across varying vineyard conditions and leaf densities.

Next, the deep learning models themselves can be further optimized. The dataset used for training and evaluating the models was limited to the PlantVillage dataset and a small set of annotated grapevine images. This dataset may not fully represent the diversity of real-world vineyard conditions. Therefore, having more comprehensive dataset that includes images captured under varying lighting conditions, occlusion levels, and disease severities would enable more robust model training and evaluation.

Additionally, the system integration aspect can be further enhanced. The current implementation focuses on the core inference pipeline, but the integration of disease detection results into the existing LoRaWAN-based monitoring system can be further refined. Besides, the system can also applied to other crops and diseases by retraining the models on relevant datasets, which would broaden the applicability of the proposed Edge AI architecture in precision agriculture.

Bibliography

- [1] M Shantkumari and SV Uma. Machine learning techniques implementation for detection of grape leaf disease. *Multimedia tools and applications*, 82(20):30709–30731, 2023.
- [2] Xu Tao, Jackson Butcher, Simone Silvestri, and Sajal K Das. icrop+: An edge-boosted crop disease detection system via tinymml and lora communication. In *2025 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*, pages 558–560. IEEE Computer Society, 2025.
- [3] Midhun P Mathew, Sajmi Salam, Sophiya Mathews, et al. Real-time automatic detection of grape leaf diseases with mobilenetv2 deep learning network based on artificial intelligence. In *2025 Advanced Computing and Communication Technologies for High Performance Applications (ACCTHPA)*, pages 1–6. IEEE, 2025.
- [4] Enyu Liu and Haiming Li. Grape leaf disease classification based on convolutional neural networks. In *2025 10th International Conference on Computer and Information Processing Technology (ISCIPT)*, pages 257–260. IEEE, 2025.
- [5] Fatma M Talaat, Mahmoud Y Shams, Samah A Gamel, and Hanaa ZainEldin. Deepleaf: an optimized deep learning approach for automated recognition of grapevine leaf diseases. *Neural Computing and Applications*, pages 1–25, 2025.
- [6] Zihan Zhou and Liming Yu. Lightweight grape leaf disease identification for complex field scenes–yolo-grape model and its boundary-lesion coalesce optimization policies: Z. zhou, l. yu. *Signal, Image and Video Processing*, 19(16):1378, 2025.
- [7] Fatih Atesoglu and Harun Bingol. The detection and classification of grape leaf diseases with an improved hybrid model based on feature engineering and ai. *AgriEngineering*, 7(7):228, 2025.
- [8] Madhuri Dharrao, Rajasmita Panda, Anupkumar Bongale, Vinod Mahajan, Sirshananda Panda, and Deepak Dharrao. Efficient grape diseases classification using an optimized mobilenetv2 with special attention block. In *2025 9th International Conference on Computing, Communication, Control and Automation (ICCCBEA)*, pages 1–6. IEEE, 2025.
- [9] Mihaela Hnatiuc, Simona Ghita, Domnica Alpetri, Aurora Ranca, Victoria Artem, Ionica Dina, Mădălina Cosma, and Mazin Abed Mohammed. Intelligent grapevine disease detection using iot sensor network. *Bioengineering*, 10(9):1021, 2023.
- [10] Prashant G Aher, Vikrant Sabnis, and Jay Kumar Jain. Deep learning for grape leaf disease detection: A review. *Multidisciplinary Reviews*, 8(11):2025364–2025364, 2025.
- [11] Maram Fahaad Almufareh, Mamoonah Humayun, Zulfiqar Ahmad, and Asfandiyar Khan. An intelligent lorawan-based iot device for monitoring and control solutions in smart farming through anomaly detection integrated with unsupervised machine learning. *IEEE Access*, 12:119072–119086, 2024.



- [12] Theresa Jose and J Albert Mayan. Lorawan and artificial intelligence integrated smart acoustic sensor network for bird species identification and deterrence system for farm protection. *Measurement*, page 118452, 2025.
- [13] Apeksha Gawande, Swati Sherekar, and Ranjit Gawande. Early prediction of grape disease attack using a hybrid classifier in association with iot sensors. *Heliyon*, 10(19), 2024.
- [14] Mihaela Hnatiuc, Domnica Alpetri, Sorin Sinte, Bogdan Hnatiuc, Gabriel Margarit Raicu, Mirel Paun, and Ionica Dina. Aiot monitoring for early identification of diseases in grapevines. complete study. *IEEE Access*, 2025.
- [15] R Karthik, R Menaka, S Ompirakash, P Bala Murugan, M Meenakashi, Sindhia Lingaswamy, and Daehan Won. Grapeleafnet: A dual-track feature fusion network with inception-resnet and shuffle-transformer for accurate grape leaf disease identification. *IEEE Access*, 12:19612–19624, 2024.
- [16] Mingji Yang, Xinbo Tong, and Haisong Chen. Detection of small lesions on grape leaves based on improved yolov7. *Electronics*, 13(2):464, 2024.
- [17] Zuxing Chen, Junjie Feng, Kun Zhu, Zhenyan Yang, Yanhong Wang, and Mingyue Ren. Yolov8-accw: Lightweight grape leaf disease detection method based on improved yolov8. *IEEE Access*, 12:123595–123608, 2024.
- [18] Apri Junaidi, Siti Zaiton Mohd Hashim, Mohd Shahizan Bin Othman, Mohd Muradha Mohamad, Hitham Alhussian, Said Jadil Abdulkadir, Maged Nasser, and Yunusa Adamu Bena. Deep learning and edge computing in agriculture: A comprehensive review of recent trends and innovations. *IEEE Access*, 2025.
- [19] Harsh Joshi. Edge-ai for agriculture: lightweight vision models for disease detection in resource-limited settings. *arXiv preprint arXiv:2412.18635*, 2024.
- [20] BJ Santhosh and C Balarengadurai. Edge-ai based plant leaf disease identification and prevention for smart agriculture. *Grenze International Journal of Engineering & Technology (GIJET)*, 10, 2024.
- [21] Md Jawadul Karim, Md Omaer Faruq Goni, Md Nahiduzzaman, Mominul Ahsan, Julfikar Haider, and Marcin Kowalski. Enhancing agriculture through real-time grape leaf disease classification via an edge device with a lightweight cnn architecture and grad-cam. *Scientific Reports*, 14(1):16022, 2024.
- [22] Carolina Lazcano-García, Karen Guadalupe García-Resendiz, Jimena Carrillo-Tripp, Everardo Inzunza-Gonzalez, Enrique Efrén García-Guerrero, David Cervantes-Vasquez, Jorge Galarza-Falfan, Cesar Alberto Lopez-Mercado, and Oscar Adrian Aguirre-Castro. Deep learning-based system for early symptoms recognition of grapevine red blotch and leafroll diseases and its implementation on edge computing devices. *AgriEngineering*, 7(3):63, 2025.
- [23] Sajib Bin Mamun, Israt Jahan Payel, Md Taimur Ahad, Anthony S Atkins, Bo Song, and Yan Li. Grape guard: A yolo-based mobile application for detecting grape leaf diseases. *Journal of Electronic Science and Technology*, 23(1):100300, 2025.

- [24] Apeksha Ranjit Gawande, Swati Sherekar, and Ranjit Madhukar Gawande. Analyzing various grape diseases and detection techniques by monitoring environmental parameters through iot and machine learning approaches. In *Machine Learning for Environmental Monitoring in Wireless Sensor Networks*, pages 173–188. IGI Global Scientific Publishing, 2025.
- [25] S Kirubakaran and Shaheed Samreen. Grape leaf diseases identification system using convolutional neural networks and lora technology. *Library of Progress-Library Science, Information Technology & Computer*, 44(3), 2024.
- [26] Seyed Mohamad Javidan, Ahmad Banakar, Keyvan Asefpour Vakilian, and Yiannis Ampatzidis. Diagnosis of grape leaf diseases using automatic k-means clustering and machine learning. *Smart Agricultural Technology*, 3:100081, 2023.
- [27] A Diana Andrushia, T Mary Neebha, A Trephena Patricia, S Umadevi, N Anand, and Atul Varshney. Image-based disease classification in grape leaves using convolutional capsule network. *Soft Computing*, 27(3):1457–1470, 2023.
- [28] Deeksha Nargotra and Vinod Sharma. Grape leaf disease detection using deep learning. *Journal of Scientific Research and Technology*, pages 128–139, 2023.
- [29] Tanishq Soni. Deep learning for accurate grape disease classification with resnet50. In *2024 5th International Conference on Smart Electronics and Communication (ICOSEC)*, pages 816–820. IEEE, 2024.
- [30] S Phani Praveen, Rajeswari Nakka, Anuradha Chokka, Venkata Nagaraju Thatha, Sai Srinivas Vellela, and Uddagiri Sirisha. A novel classification approach for grape leaf disease detection based on different attention deep learning techniques. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 14(6):2023, 2023.
- [31] Hoang-Tu Vo, Kheo Chau Mui, and Nhon Nguyen Thien. Fusing xception and resnet50 features for robust grape leaf disease classification. *Bulletin of Electrical Engineering and Informatics*, 14(1):268–275, 2025.
- [32] Deepshikha Yadav, Archana Balyan, Suman Mann, and Aayush Ranga. Enhancing disease detection prediction accuracy of grape leaves using vgg16 model and inception v3 model. *Proceedings on Engineering*, 7(1):365–370, 2025.
- [33] Sarah Khursheed and Balwinder Kaur. Improved grape leaf disease detection with gan-augmented resnet and efficientnet. In *International Conference on Advances and Applications in Artificial Intelligence (ICAAAI 2025)*, pages 654–669. Atlantis Press, 2025.
- [34] Ning Zhang, Enxu Zhang, Guowei Qi, Fei Li, and Cheng Lv. Lightweight grape leaf disease recognition method based on transformer framework. *Scientific Reports*, 15(1):28974, 2025.
- [35] Weixia Li, Bingkun Zhou, Yinzheng Zhou, Chenlu Jiang, Mingzhuo Ruan, Tangji Ke, Huijun Wang, and Chunli Lv. Grape disease detection using transformer-based integration of vision and environmental sensing. *Agronomy*, 15(4):831, 2025.

- [36] Xinxin Zhang, Chaojun Cen, Fei Li, Meng Liu, and Weisong Mu. Crformer: cross-resolution transformer for segmentation of grape leaf diseases with context mining. *Expert Systems with Applications*, 229:120324, 2023.
- [37] Farian S Ishengoma and Neema N Lyimo. Ensemble model for grape leaf disease detection using cnn feature extractors and random forest classifier. *Heliyon*, 10(12), 2024.
- [38] Yiping Chen and Qiufeng Wu. Grape leaf disease identification with sparse data via generative adversarial networks and convolutional neural networks. *Precision Agriculture*, 24(1):235–253, 2023.
- [39] Changpeng Ji, Yongji Zuo, and Wei Dai. Research on grape leaf disease detection method based on nma-yolov8n. In *ITM Web of Conferences*, volume 77, page 01038. EDP Sciences, 2025.
- [40] Shiva Mehta, Vinay Kukreja, and Dibyahash Bordoloi. Grape leaf disease severity analysis: Employing federated learning with cnn techniques. In *2023 World Conference on Communication & Computing (WCONF)*, pages 1–6. IEEE, 2023.
- [41] Poonam Gaur, Rohit Sharma, Ravi Kumar, Amit Gupta, Rishabh Sharma, and Vinay Kukreja. Fighting grape black rot with deep learning: A cnn-lstm hybrid model for disease severity classification. In *2023 4th International Conference for Emerging Technology (INCET)*, pages 1–5. IEEE, 2023.
- [42] Ismail Kunduracioglu and Ishak Pacal. Advancements in deep learning for accurate classification of grape leaves and diagnosis of grape diseases. *Journal of Plant Diseases and Protection*, 131(3):1061–1080, 2024.
- [43] Varsha Naik, Trupti Baraskar, Viren Khatri, Parimal Kolhe, and Mitheesh Katyarmal. Disease detection in grape leaves using. In *Advances in Intelligent Systems for Sustainable Agriculture: Proceedings of the 3rd International Conference, ICISA 2024*, volume 1201, page 3. Springer Nature, 2025.
- [44] Savitha Shetty and Saritha Shetty. Grape leaf disease detection using deep learning approach. In *AIP Conference Proceedings*, volume 3278, page 020006. AIP Publishing LLC, 2025.
- [45] Jianwu Lin, Xiaoyulong Chen, Renyong Pan, Tengbao Cao, Jitong Cai, Yang Chen, Xishun Peng, Tomislav Cernava, and Xin Zhang. Grapenet: A lightweight convolutional neural network model for identification of grape leaf diseases. *Agriculture*, 12(6):887, 2022.
- [46] Ahmad Nur Sahid and Deden Ruli Cahyadi. Image classification using mobilenet based on cnn architecture for grape leaf disease detection. *Journal of Intelligent Systems Technology and Informatics*, 1(1):15–21, 2025.
- [47] Kazi Tanvir, Ritik Sharma, Md Sayem Kabir, Tasnim Sultana Sintheia, Soumya Basu, and Soumik Banerjee. Pcnnet-based grape leaf disease detection using mobilenetv2 and vit. *Data Science and Security: Proceedings of IDSCS 2024, Volume 1*, 1:281, 2025.



- [48] R Deepa, Midhun P Mathew, S Baskar, et al. L-net: a lightweight cnn framework for sustainable multicrop leaf disease detection and classification on edge devices. *Sustainable Food Technology*, 2026.
- [49] Milan Grujev, Dragan Stojanovic, Aleksandar Milosavljevic, Milos Ilic, and Veljko Prodanovic. An efficient real-time apple disease classification approach using a novel lightweight neural network on an arduino edge device. *Internet of Things*, page 101833, 2025.
- [50] Gulshat Amirkhanova, Syrym Ismailov, Alikhan Amirkhanov, Saltanat Adilzhanova, Meiramkul Zhasuzakova, and Siming Chen. A lightweight, end-to-end encrypted data pipeline for iiot: An aes-gcm implementation for esp32, mqtt, and raspberry pi. *Information*, 17(1):33, 2026.
- [51] Espressif Systems. Esp32-s3 datasheet, 2022.
- [52] Abdallah Ali. Plantvillage dataset. Kaggle, 2023. Accessed: February 2026.
- [53] Nicolaas Regnier. Grape / grapes. Kaggle, 2023. Accessed: February 2026.
- [54] Roboflow Universe. Grape leaf roi detection dataset, version 6. Roboflow, 2026. CC BY 4.0 License, Exported: January 29, 2026 from Roboflow Universe at <https://universe.roboflow.com/grape-disease-rxjfx/grape-leaf-roi-detection/dataset/6>.
- [55] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, pages 91–99, 2015.
- [56] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [57] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. Ssd: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pages 21–37, 2016.
- [58] Glenn Jocher and Jing Qiu. Ultralytics yolo11, 2024.
- [59] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.
- [60] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.
- [61] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(9):1904–1916, 2015.



- [62] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [63] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.
- [64] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-iou loss: Faster and better learning for bounding box regression. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(07):12993–13000, 2020.
- [65] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: ommon objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.
- [66] Espressif Systems. Esp-ppq: Post-training quantization for esp-dl. <https://github.com/espressif/esp-ppq>, 2023. Accessed: February 2026.
- [67] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.
- [68] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [69] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255. IEEE, 2009.
- [70] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3320–3328, 2014.
- [71] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [72] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018.
- [73] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018.
- [74] Freenove. Freenove esp32-s3-wroom development board documentation, 2024. Official hardware documentation including specifications, schematics, and example code for camera, WiFi, and sensor integration.
- [75] Espressif Systems. Esp-dl: Espressif deep learning library, 2024.



- [76] Espressif Systems. How to deploy MobileNetV2 on ESP32 — model quantization, 2024. ESP-DL Official Documentation.
- [77] Thomas G. Dietterich, Richard H. Lathrop, and Tomás Lozano-Pérez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1-2):31–71, 1997.
- [78] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [79] OmniVision Technologies. Ov03660-a51a datasheet. http://datasheet.iiic.cc/datasheets-0/omnivision_technologies/OV03660-A51A.pdf, 2025. Accessed: 2025.
- [80] Espressif Systems. Esp32-s3-wroom-1 datasheet. https://documentation.espressif.com/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf, 2025. Accessed: 2025.



Inference Results on ESP32-S3

The following listing presents a representative inference cycle captured from the ESP32-S3 serial monitor. The output covers the full boot sequence, hardware initialisation, model loading, image capture, object detection, per-leaf disease classification, weighted aggregation, and the final diagnosis.

```
===== BOOT =====
Build: Jun 27 2026 14:30:00
rst:0x1 (POWERON), boot:0x9 (SPI_FAST_FLASH_BOOT)
ESP-IDF v5.3.3 | Chip rev: v0.2 | Multicore bootloader
SPI Mode: QIO | Flash: 16 MB | Boot Speed: 80 MHz

PSRAM: 8 MB (AP, Gen 3, 64 Mbit, 80 MHz, Octal)
CPU freq: 240 MHz | Multicore app
PSRAM memory test: OK

===== SYSTEM INFORMATION =====
Chip:      ESP32-S3
Cores:     2
Silicon:   Rev 2
Flash:     16 MB external
Camera:    OV3660 (640x480)
Free Heap: 8,638,632 bytes
Free PSRAM: 8,385,720 bytes

===== MODEL INITIALISATION =====
[1] Detection model initialised      ... 144 ms
    Free heap after init: 7,095 KB

[2] Disease classifier (MobileNetV2 128x128)
    Model: mobilenetv2_128_grape_leaf.espd1
    Location: FLASH_RODATA
    Packed binary size: 2,863,072 bytes (2.73 MB)
    Input shape: [1, 128, 128, 3]
    Input buffer: 49,152 bytes (PSRAM)
    Initialised in      ... 348 ms
    Free heap after init: 4,207 KB

[3] Camera (OV3660)
    I2C SCCB: SDA=4, SCL=5 (port 1)
    PID: 0x3660 | Address: 0x3C
    PCLK: 10 MHz | Format: JPEG | Quality: 12
    Frame buffers: 2 x 61,440 bytes (PSRAM)

===== FRAME 1 =====
Capture: 640x480, 18,495 bytes      78 ms
Decode:  640x480 RGB888            55 ms
Detection: 10 objects found        358 ms

----- Per-Leaf Classification -----
Leaf  Bbox [x1,y1,x2,y2]  Area  Conf  Top Disease  Prob
[0]   [192,201,280,243]   3,696  0.500  Leaf_blight  47.25%
[1]   [123,278,270,401]  18,081  0.500  Healthy      69.59%
[2]   [557,  0,611, 28]   1,512  0.321  Healthy      48.62%
[3]   [599,  0,639, 53]   2,120  0.321  Healthy      65.15%
[4]   [341, 43,436,101]   5,510  0.321  Healthy      69.16%
[5]   [376,228,462,354]  10,836  0.321  Leaf_blight  44.10%
[6]   [163, 13,249, 65]   4,472  0.269  Healthy      56.02%
[7]   [520,  0,579, 50]   2,950  0.269  Healthy      50.31%
[8]   [322, 41,484,103]  10,044  0.269  Healthy      53.57%
[9]   [514, 46,638,134]  10,912  0.269  Leaf_blight  60.95%

----- Leaf Weights -----
Leaf  Weight  Entropy  Certainty  BBox Qual
[0]   0.0082  0.895    10.5%      0.157
[1]   0.0925  0.619    38.1%      0.486
```



```
[2]  0.0004  0.829  17.1%  0.008
[3]  0.0054  0.728  27.2%  0.062
[4]  0.0217  0.658  34.2%  0.198
[5]  0.0264  0.833  16.7%  0.493
[6]  0.0092  0.766  23.4%  0.146
[7]  0.0084  0.821  17.9%  0.175
[8]  0.0054  0.811  18.9%  0.106
[9]  0.0107  0.740  26.0%  0.153

--- Weighted Class Scores (S_c) -----
Black_rot:      8.17%
Esca:           5.00%
Healthy:        58.22%
Leaf_blight:    28.62%

===== FINAL DIAGNOSIS =====
Disease:        Healthy
Confidence:      58.2%
Leaves:          10 analysed
Total Weight:    0.1883

===== PERFORMANCE =====
Capture:         78 ms
Decode:          55 ms
Detection:        358 ms
Disease:         5,898 ms (classification pipeline)
TOTAL:           6,389 ms (0.16 FPS)
Free PSRAM:      3,910 KB

>> Sleeping for 6 hours
```

Listing 1: Complete inference cycle on ESP32-S3 (serial monitor output).



Model Packaging Script

The following CMake script is responsible for packing the two `.espd1` model files—the ESPDet-Pico object-detection model and the MobileNetV2 disease-classification model—into a single binary blob that is embedded in the ESP32-S3 flash at build time. The ESP-DL `pack_espd1_models.py` utility concatenates both models into one file (`grape_leaf_detect.espd1`), which is then linked into the firmware as aligned read-only data in the `FLASH_RODATA` segment.

```
1 # ---- Locate the ESP-DL component directory ----
2 idf_build_get_property(component_targets __COMPONENT_TARGETS)
3 if ("__idf_espressif__esp-dl" IN_LIST component_targets)
4     idf_component_get_property(espd1_dir espressif__esp-dl
5                               COMPONENT_DIR)
6 elseif("__idf_esp-dl" IN_LIST component_targets)
7     idf_component_get_property(espd1_dir esp-dl COMPONENT_DIR)
8 endif()
9 set(cmake_dir ${espd1_dir}/fbs_loader/cmake)
10 include(${cmake_dir}/utilities.cmake)
11
12 # ---- Component registration ----
13 set(src_dirs .)
14 set(include_dirs .)
15 set(requires esp-dl)
16 set(packed_model ${BUILD_DIR}/espd1_models/grape_leaf_detect.espd1)
17
18 idf_component_register(
19     SRC_DIRS      ${src_dirs}
20     INCLUDE_DIRS  ${include_dirs}
21     REQUIRES      ${requires})
22
23 # ---- Pack detection + classification models ----
24 if (NOT CONFIG_ESPDET_DETECT_MODEL_IN_SDCARD)
25     if (IDF_TARGET STREQUAL "esp32s3")
26         set(models_dir ${COMPONENT_DIR}/models/s3)
27     elseif (IDF_TARGET STREQUAL "esp32p4")
28         set(models_dir ${COMPONENT_DIR}/models/p4)
29     endif()
30
31     file(MAKE_DIRECTORY ${BUILD_DIR}/espd1_models)
32     set(models)
33
34     # 1. Object-detection model (ESPDet-Pico 416x320)
35     if(CONFIG_FLASH_ESPDET_PICO_416_320_GRAPE_LEAF)
36         list(APPEND models
37             ${models_dir}/espdet_pico_416_320_grape_leaf.espd1)
38     endif()
39
40     # 2. Disease-classification model (MobileNetV2 128x128)
41     list(APPEND models
42         ${models_dir}/mobilenetv2_128_grape_leaf.espd1)
43
44     # Pack both models into a single binary
45     set(pack_model_exe
```



```
45     ${espd1_dir}/fbs_loader/pack_espd1_models.py)
46 add_custom_command(
47     OUTPUT    ${packed_model}
48     COMMENT   "Move and Pack models..."
49     COMMAND   python ${pack_model_exe}
50               --model_path ${models}
51               --out_file    ${packed_model}
52     DEPENDS   ${models}
53     VERBATIM)
54
55 # Embed in flash as aligned read-only data
56 if(CONFIG_ESPDET_DETECT_MODEL_IN_FLASH_RODATA)
57     target_add_aligned_binary_data(
58         ${COMPONENT_LIB} ${packed_model} BINARY)
59 endif()
60
61 # Or flash to a dedicated partition
62 if(CONFIG_ESPDET_DETECT_MODEL_IN_FLASH_PARTITION)
63     message(STATUS
64         "Model will be flashed to espdet_det partition separately")
65 endif()
66 endif()
```

Listing 2: CMake component script for packing and embedding the detection and classification models.



Quick Links

- [Go to Abstract](#)
- [Go to Table of Contents](#)
- [Go to Introduction](#)
- [Go to Literature Review](#)
- [Go to System Architecture and Model Development](#)
- [Go to Edge AI system Implementation](#)
- [Go to Performance Evaluation and Discussion](#)
- [Go to Conclusion and Future Work](#)
- [Go to References](#)